



Improving Insight Quality in Amazon App Review Through Sentiment Analysis and Anomaly Detection

Module Title: Interdisciplinary Group Project

Module Code: UFCFWQ-45-M

Module Leader: Paul Mathews

Team 11

Members Name

Pranika Shrestha - 25059421

Subhashree Thapa - 25034740

Puja Gohel - 25020509

Ajay Bhuj - 25051512

Aharan Elankovan - 25018524

Submission Date: 30th April, 2026

Table of Contents

1. Introduction:	1
1.1 Background & Problem Context:	1
1.2 Aims and Objectives:	1
1.3 Scope:	2
2. Annotated Literature Review:	2
3. Project Management and Team Participation:	3
3.1 Task allocation and contributions	3
3.2 Tools used:	4
4. Research and Development Documentation:	4
4.1 Phase 1 — Exploratory Data Analysis (EDA)	5
4.1.1 Data Loading & Quality Assessment	5
4.1.2 Data Cleaning & Preprocessing	6
4.1.3 Feature Engineering (26 Numeric Features)	6
4.1.4 Key Findings & Visualisations	7
4.1.5 Output Datasets	10
4.2 Phase 2 — Fake Review Detection	10
4.2.1 Approach Overview	10
4.2.2 Rule-Based Detection (10 Heuristic Rules)	10
4.2.3 Isolation Forest — Anomaly Detection	12
4.2.4 Contamination Sensitivity Analysis	12
4.2.5 Ensemble Voting & Threshold Selection	13
4.2.6 Validation & Results	14
4.3 Phase 3 — Sentiment Analysis	16
4.3.1 Data Preparation & TF-IDF Vectorisation	16
4.3.2 Model Training (Logistic Regression & Naive Bayes)	17
4.3.3 Model Comparison & Evaluation	17
4.3.4 Impact Analysis — Effect of Fake Review Removal	20

4.4 Phase 4 — Interactive Dashboard	21
4.4.1 Architecture	21
4.4.2 Page 1 — Project Overview	21
4.4.3 Page 2 — Fake Review Detection	21
4.4.4 Page 3 — Sentiment Analysis	22
4.4.5 Page 4 — Live Prediction	22
5. Problems Encountered and Solutions:	22
6. Team Meeting Minute:	22
7. Prototypes, Visualisations and Evaluation:	22
7.1 Dashboard Prototype:	22
7.2 Exploratory Data Analysis Output:	23
7.3 Anomaly Detection (Fake Review Detection):	30
7.3.1 Output and Visualisation:	30
7.3.2 Evaluation:	32
7.4 Sentiment Analysis:	33
7.4.1 7.4.1 Output and Visualisation:	33
7.4.2 7.4.2 Evaluation:	37
7.5 Integrated System Evaluation:	38
8. Sprint Retrospectives:	40
9. Code and Technical Artefacts:	40
9.1 Link to repositories:	40
10. Reflection:	41
10.1 Group Reflection:	41
10.2 Individual Reflection (Pranika):	41
11. Conclusion:	42
12. References:	43
13. Appendices:	44

Table of Figures:

Figure 1: 4.1 Score distribution of Amazon app reviews	5
Figure 2: 4.1 Duplicate review analysis	6
Figure 3: 4.3 Correlation heatmap of engineered features	7
Figure 4: 4.4 Word count distribution	8
Figure 5: 4.5 Thumps up distribution	8
Figure 6: 4.6 Temporal patterns in review posting	8
Figure 7: 4.7 VADER Sentiment analysis by rating	9
Figure 8: 4.8 Outlier Analysis	9
Figure 9: 4.9 Class imbalance analysis	9
Figure 10: 4.10 Per-rile fire rates	11
Figure 11: 4.11 Isolation forest feature importance	12
Figure 12: 4.12 Contamination sensitivity analysis	13
Figure 13: 4.13 Ensemble vote distribution	14
Figure 14: 4.14 Inter-method agreement heatmap.....	15
Figure 15: 4.15 PCA - Fake vs Clean Reviews	15
Figure 16: 4.16 Fake vs Real review characteristics	16
Figure 17: 4.17 Sentiment distribution	17
Figure 18: 4.18 Model comparison	18
Figure 19: 4.18 Confusion matrices	18
Figure 20: 4.20 ROC Curves.....	19
Figure 21: 4.21 5-fold cross-validation results.....	19
Figure 22: 4.22 Top words per sentiment class	20
Figure 23: 4.23 Word clouds - Positive vs Negative.....	20
Figure 24: 4.24 Impact of fake reviews removal on sentiment accuracy	21
Figure 25: 7.1 Dashboard display	23
Figure 26: EDA Output - Rating distribution of Amazon app reviews	24
Figure 27: EDA Output - Word count by score	25
Figure 28: EDA output - Word count distribution	25
Figure 29: EDA Output - Thumbs-up count distribution	26
Figure 30: EDA Output - Duplicate text flag and character length	26
Figure 31: EDA Output - Reviews per day over time	27
Figure 32: EDA Output - VADER compound by start rating and score.....	27

Figure 33: EDA Output - Top 15 app versions	28
Figure 34: EDA Output - Feature correlation heatmap.....	29
Figure 35: EDA Output (IQR Method).....	30
Figure 36: Anomaly Detection Output - IF detection result	30
Figure 37: Anomaly Detection Output - Isolation forest contamination sensitivity analysis	31
Figure 38: Anomaly Detection Output - Rule-based detection per rule fire rates.....	31
Figure 39: Anomaly Detection Output - Ensemble voting results for suspicious review detection	32
Figure 40: Sentiment Analysis - Output: Sentiment class distribution after neutral review removal.....	33
Figure 41: Sentiment Analysis - Output: Binary sentiment distribution after fake removal	34
Figure 42: Sentiment Analysis - Output: Comparing model performance	34
Figure 43: Sentiment Analysis - Output: 5-fold cross validation analysis.....	35
Figure 44: Matrices for Logistic Regression and Multinomial Naive Bayes: Sentiment Analysis - Output: Confusion.....	35
Figure 45: Sentiment Analysis Output: ROC curved for sentiment classification models	36
Figure 46: Sentiment Analysis Output: Word cloud for positive and negative reviews	36
Figure 47: Sentiment Analysis Output: Feature importance on top 20 positive and negative words.....	37
Figure 48: Sentiment distribution analysis.....	38
Figure 49: Evaluation: Impact of fake review removal on sentiment classification ...	39

Table of Tables

Table 1: Table of Star rating.....	5
Table 2: Table of Rule-based detection.....	10
Table 3: Table of Contamination sensitivity analysis	12
Table 4: Table of Ensemble voting and threshold selection	13
Table 5: Table of Model comparison	17
Table 6: Table of Sprint retrospectives.....	40

1. Introduction:

1.1 Background & Problem Context:

Online marketplaces have become important part of modern retailing, with user-generated reviews strongly influencing customer decision-making and shaping how customers view the product. Reviews provide valuable insights into customer experience, and research shows that a vast majority of online shoppers consult ratings and written feedback before making a purchase decision, highlighting their importance in establishing trust and guiding purchasing behaviour (Du, et al., 2019). However, the usefulness of these reviews can be reduced when the data contains fake reviews, repeated comments, low-quality text, sentiment-rating mismatches, or unusually biased feedback.

Amazon app reviews provide useful customer opinions, but they are also noisy and unstructured. Some reviews are very short, some contain repeated wording, and others may show unusual patterns that make them less reliable for decision-making. If these issues are not considered, businesses and customers may make decisions based on misleading review data.

This project addressed these gaps by developing a data-driven analytical framework for Amazon app reviews. The project combined exploratory data analysis, anomaly detection, sentiment analysis, and visualisation to improve the quality of insights gained from review data. By identifying suspicious review behaviour and classifying review sentiment, the project aimed to support a more reliable understanding of customer feedback.

1.2 Aims and Objectives:

Aims:

The aim of this project was to develop an analytical system for improving insight quality in Amazon app reviews by combining sentiment analysis with fake review/anomaly detection.

Objectives:

1. To collect and preprocess a dataset from Amazon user reviews and conduct exploratory data analysis (EDA) to uncover patterns.
2. To develop an unsupervised anomaly detection system to detect unusual review behaviour.
3. To implement a sentiment analysis model to classify positive, neutral, and negative sentiments.
4. To design an interactive dashboard that supports real-time analysis by allowing users to input custom text and receive instant sentiment classification and review detection.

1.3 Scope:

The scope of this project included:

- Cleaning and preparing an Amazon app review dataset for analysis.
- Conducting exploratory data analysis to understand rating patterns, review length, duplicate text, user engagement, and review activity over time.
- Engineering behavioural and linguistic features such as word count, punctuation use, duplicate text weight, rating behaviour, and VADER sentiment score.
- Applying anomaly detection to identify suspicious review patterns using Isolation Forest, rule-based detection, and ensemble voting.
- Developing sentiment analysis models to classify reviews as positive or negative.
- Comparing Logistic Regression and Multinomial Naive Bayes using accuracy, F1-score, ROC AUC, and cross-validation.

2. Annotated Literature Review:

Existing studies on sentiment analysis have shown user reviews influencing customer perception and purchasing behaviour. Earlier studies apply traditional methods such as TF-IDF and logistic regression, while modern approaches use transformer-based models like BERT to improve contextual understanding. (Holzing, et al., 2025) Although, existing approaches still struggle with informal language, unclear expressions, and noisy text which are commonly found in online reviews.

Secondly, the project focuses on detecting fake or anomalous reviews. Research emphasizes patterns such as high posting frequency, repetitive language, and sentiment-rating inconsistencies as indicators of potential spam. (Liu & Jindal, 2008) Unsupervised models including isolation forest. Challenges such as differentiating genuine variation in user behaviour from unusual behaviours arises despite all the progress.

While many studies address sentiment analysis and anomaly detection individually, few integrate these into one single analytical framework. This project addresses this gap by combining sentiment modelling and anomaly detection within an interactive dashboard.

The detailed information on Annotated Literature Review supporting our project is provided in a separate document, available via the project repository. ([GitLab Link](#))

3. Project Management and Team Participation:

The project followed an iterative Agile methodology using a bi-weekly scrum approach, allowing the team to work in structured sprints while maintaining flexibility throughout development.

The work was divided into multiple phases, including:

- Data collection and preprocessing
- Exploratory data analysis (EDA)
- Anomaly detection
- Sentiment analysis
- Dashboard development
- Documentation (Project Artefact)

Each sprint focuses on a specific stage of the project allowing the team to:

- Work incrementally
- Continuously improve outputs
- Adapt based on feedback and research findings

Regularly weekly meetings were conducted to:

- Review progress
- Discuss challenges
- Plan and assign upcoming tasks

The project management approach evolved based on insights gained during development. Task tracking and workflow management were supported using a [Trello](#) board, where tasks were organised in biweekly phases and moved from planning to completion. A [Gantt chart](#) was also used to provide a high-level timeline of the project and is available via the repository.

3.1 Task allocation and contributions

Tasks were distributed on both roles and technical expertise, ensuring balanced contributions across the team. This structured allocation allowed each member to take ownership of specific components while also supporting collaborative work when required.

The detailed information on Task allocation and individual contributions is provided in a separate document, available via the project repository. ([GitLab Link](#))

3.2 Tools used:

The team used a combination of tools for development, management, and collaboration:

Project Management Tool:

Trello: Used to track weekly tasks and track progress across project phases

Gantt Chart: Used for long-term timeline planning and monitoring the overall project timeline

Development Tools:

Python (Jupyter Notebook): Used for data cleaning, EDA, and model development

Pandas & NumPy: Used for data manipulation and processing

TF-IDF Vectorisation: Used for text feature extraction in sentiment analysis

Dashboard:

Streamlit / Dash: Used to develop the interactive dashboard for real-time predictions.

Plotly: Used to make interactive dashboard

Collaboration Tools:

GitLab: Used for version control and code sharing.

Google Drive: Used for sharing documents and project files.

Meeting Minutes: Used to record weekly discussions and track decisions.

4. Research and Development Documentation:

This chapter presents the three-phase analytical pipeline that was designed to analyze reviews of Amazon apps. The pipeline progresses through three phases: exploratory data analysis (Phase 1), through fake review detection (Phase 2), to sentiment classification (Phase 3) which forms a sequential data processing workflow.

4.1 Phase 1 — Exploratory Data Analysis (EDA)

The first phase focused on understanding the structure, quality, and patterns within the Amazon App Reviews dataset. All analysis was conducted in the Jupyter notebook Code/[EDA.ipynb](#).

4.1.1 Data Loading & Quality Assessment

The raw Amazon App Reviews dataset was loaded and inspected for quality issues. The dataset contained 82,069 reviews with 32 features after initial processing.

The score distribution revealed that the class is highly imbalanced as 45.4% of reviews were 1-star ratings and 29.4% were 5-star ratings, with the remaining 25.2% spread across 2–4 stars. This pattern is common in app review dataset as users tend to leave a review only when they are highly satisfied or dissatisfied.

Table 1: Table of Star rating

Rating	Count	Percentage
1 star	37,285	45.4%
2 stars	8,642	10.5%
3 stars	6,709	8.2%
4 stars	5,334	6.5%
5 stars	24,099	29.4%

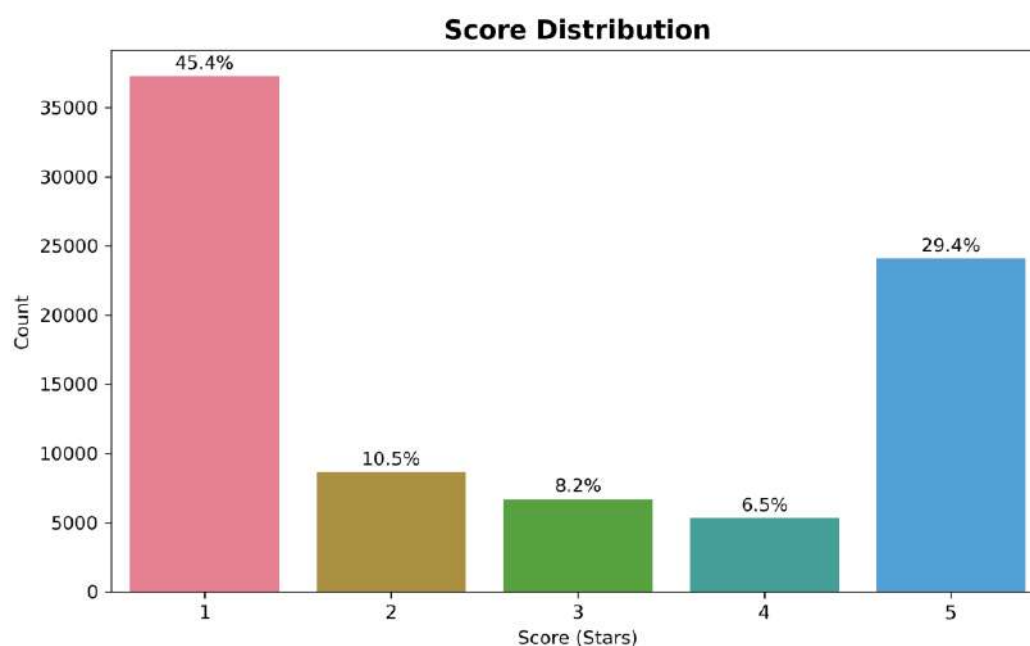


Figure 1: 4.1 Score distribution of Amazon app reviews

4.1.2 Data Cleaning & Preprocessing

For the downstream analysis, several steps were taken to prepare the data. Different temporal features were extracted such as Year, Month, Day_of_week, and hour. The duplicate reviews were removed. The raw review text was cleaned through lowercasing and punctuation normalisation to produce a content_clean column used in subsequent text analysis.

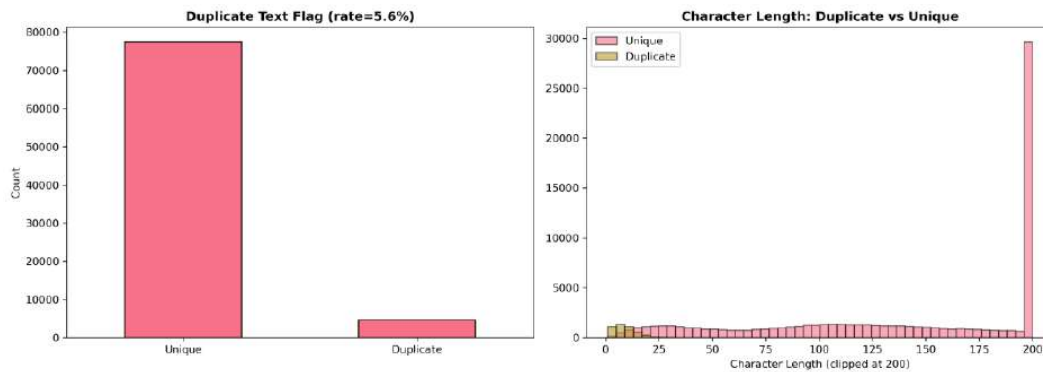


Figure 2: 4.1 Duplicate review analysis

4.1.3 Feature Engineering (26 Numeric Features)

A total of 26 numeric features were engineered from the raw text and metadata to capture various dimensions of review behaviour. These features were designed to support both anomaly detection in Phase 2 and general pattern analysis. The features fall into the following categories:

Text length features: word_count, char_len, and avg_word_len are used to capture the substance and complexity of each review.

Punctuation and symbol features: exclamation_count, question_count, caps_ratio, and punctuation_ratio are used to understand the writing style patterns that may indicate spam or manipulation.

Behavioural features: emoji_count, has_url, and repeated_char_flag are used to detect informal or promotional content.

User-level features: is_anonymous_user and reviews_by_username can be used to identify reviewer profiles and prolific posting behaviour.

Lexical features: unique_word_ratio and stopword_ratio assess vocabulary diversity and content substance.

Rating-derived features: score_extremity is used to measure the distance from the neutral rating (3 stars).

Duplicate detection: is_duplicate_text and dup_text_weight flag copy-paste reviews with length-weighted scoring.

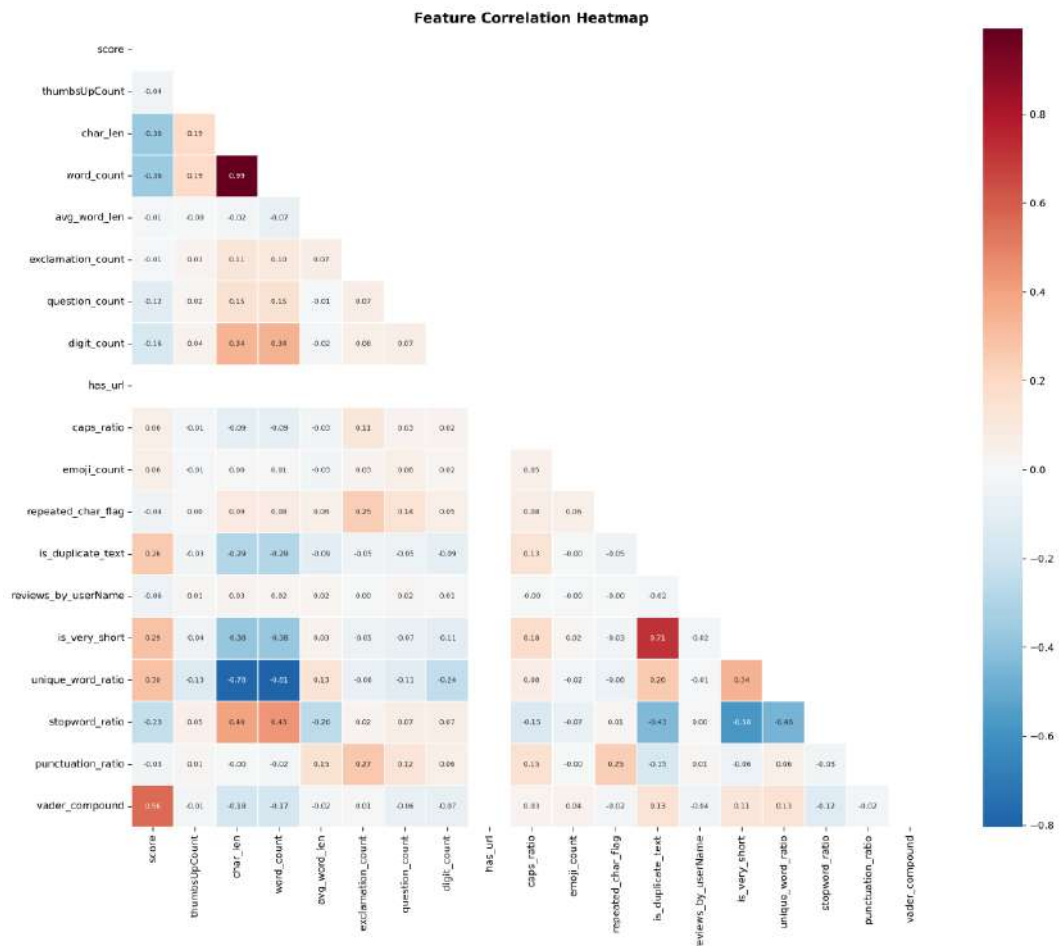


Figure 3: 4.3 Correlation heatmap of engineered features

4.1.4 Key Findings & Visualisations

The exploratory analysis provided several important insights. The word count distribution revealed that most reviews are quite short, with a median of 27 words. However, some reviews exceed 300 words. The thumbs-up distribution was heavily right-skewed. Most reviews received no helpful votes, while a small number received thousands. Temporal analysis showed patterns in when reviews were posted. Certain hours and weekdays had higher activity. The correlation heatmap showed relationships between engineered features. There was a strong correlation between word count and character length, and between score and VADER compound. The VADER sentiment analysis indicated a general agreement between star ratings and

sentiment scores based on the lexicon. However, there were notable mismatches, especially with extreme ratings.

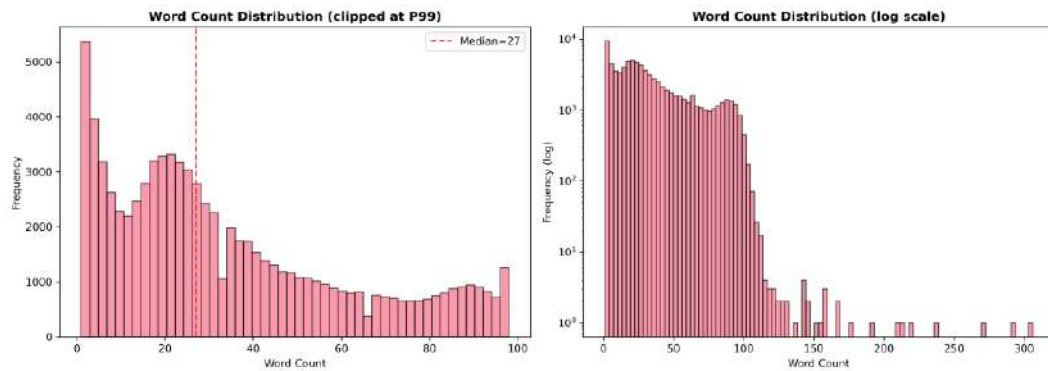


Figure 4: 4.4 Word count distribution

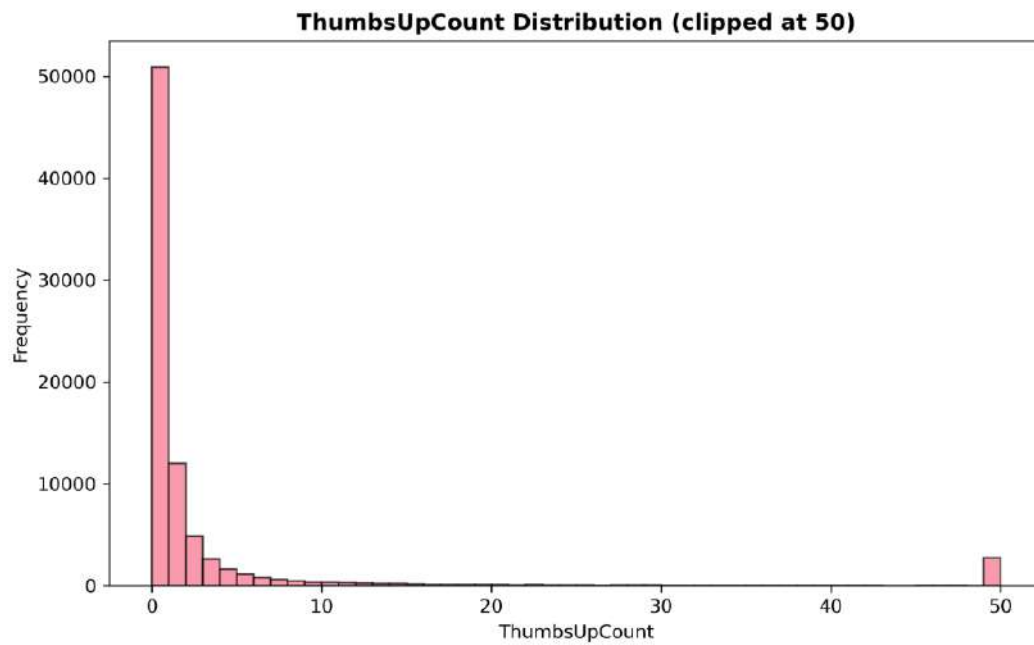


Figure 5: 4.5 Thumbs up distribution

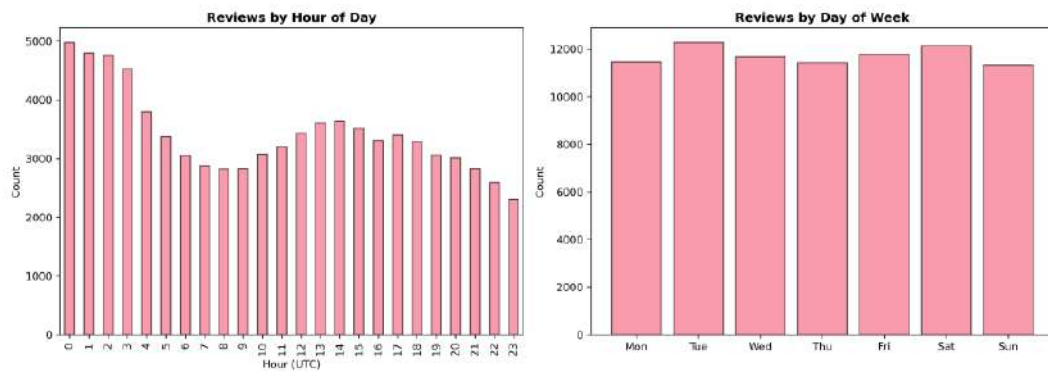


Figure 6: 4.6 Temporal patterns in review posting

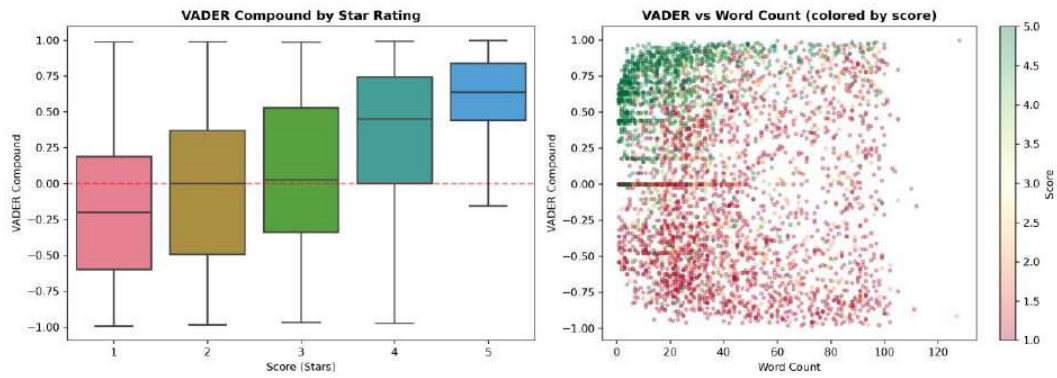


Figure 7: 4.7 VADER Sentiment analysis by rating

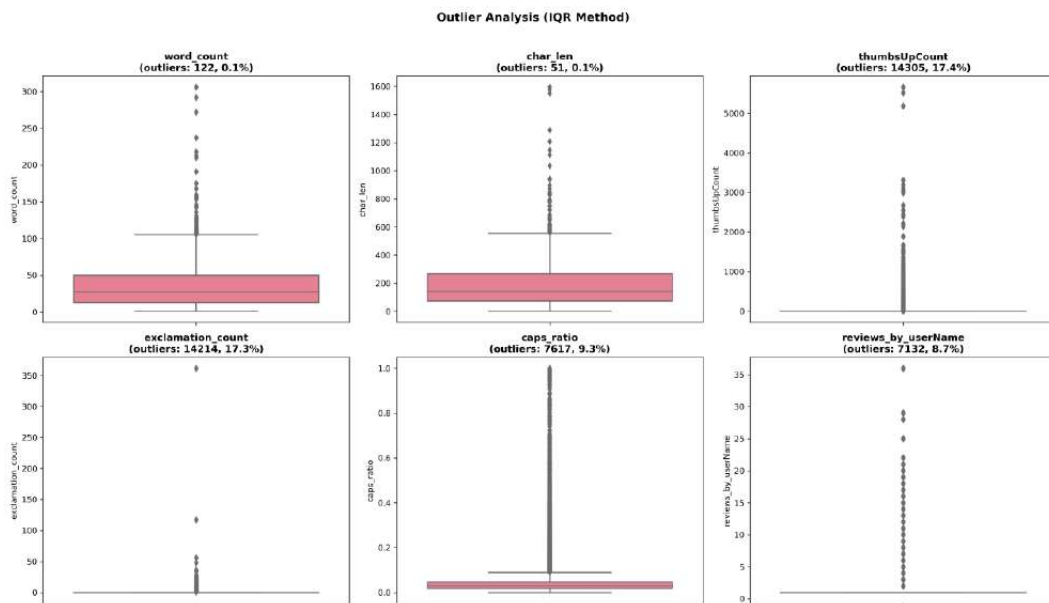


Figure 8: 4.8 Outlier Analysis

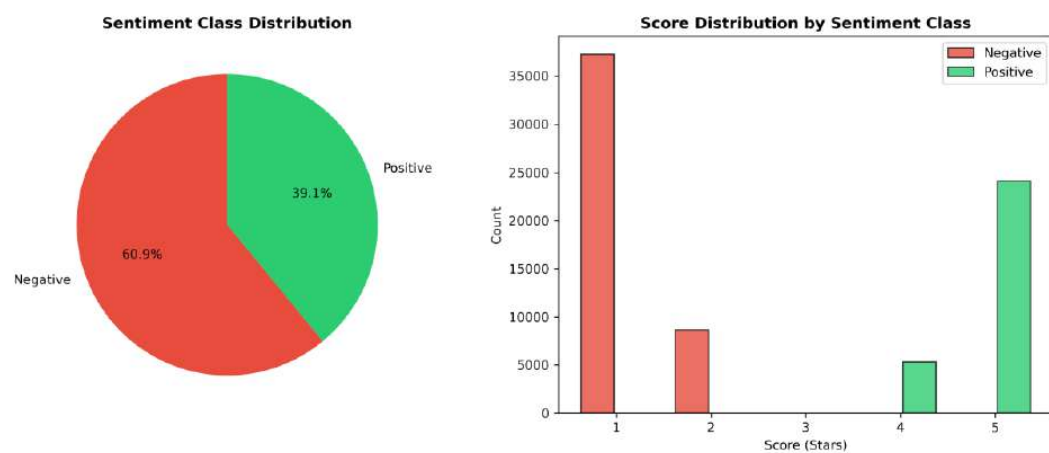


Figure 9: 4.9 Class imbalance analysis

4.1.5 Output Datasets

Two output datasets were produced. The file `reviews_clean_all.csv`, which has 82,069 rows and 32 features, contains all reviews with processed features. This file went to Phase 2 for fake review detection. The file `reviews_clean_full.csv` leaves out neutral (3-star) reviews and includes binary sentiment labels. This serves as another option for direct sentiment analysis.

4.2 Phase 2 — Fake Review Detection

The second phase implemented a hybrid fake review detection system combining rule-based heuristics with unsupervised machine learning. All work was conducted in [“isolationForest.ipynb”](#).

4.2.1 Approach Overview

The detection pipeline was designed with three components: (A) ten rule-based heuristic voters, each targeting a specific pattern of suspicious reviews; (B) an Isolation Forest anomaly detector operating on 21 numeric features; and (C) an ensemble voting system that combines all 11 voters to produce a final fake/genuine classification. This hybrid approach balances interpretability (rules) with the ability to detect complex, non-obvious anomalies (Isolation Forest).

4.2.2 Rule-Based Detection (10 Heuristic Rules)

Ten independent binary rules were defined, each casting one vote when triggered. These rules target common patterns of fake or low-quality reviews identified during EDA:

Table 2: Table of Rule-based detection

Rule	Description	Flagged	%
R1	Generic short review + extreme rating (≤ 3 words, 1 or 5 stars)	6,564	8.00%
R2	Extreme rating + no thumbs up + short (< 10 words)	13,192	16.07%
R3	Excessive capitalisation ($> 50\%$, reviews > 20 chars)	467	0.57%

R4	Weighted duplicate text (dup_text_weight ≥ 0.5)	99	0.12%
R5	Excessive exclamation marks (>5)	984	1.20%
R6	Many emojis (>3)	132	0.16%
R7	VADER-score mismatch (sentiment contradicts rating)	5,171	6.30%
R8	Low lexical diversity (unique_word_ratio <0.4, >10 words)	3	0.00%
R9	Contains URL (promotional/spam)	0	0.00%
R10	Short anonymous review + extreme rating	10	0.01%

In total, 19,646 reviews (23.94%) were flagged by at least one rule. Rule R2 had the highest fire rate at 16.07%, reflecting the large number of short, extreme-rated reviews with no community engagement.

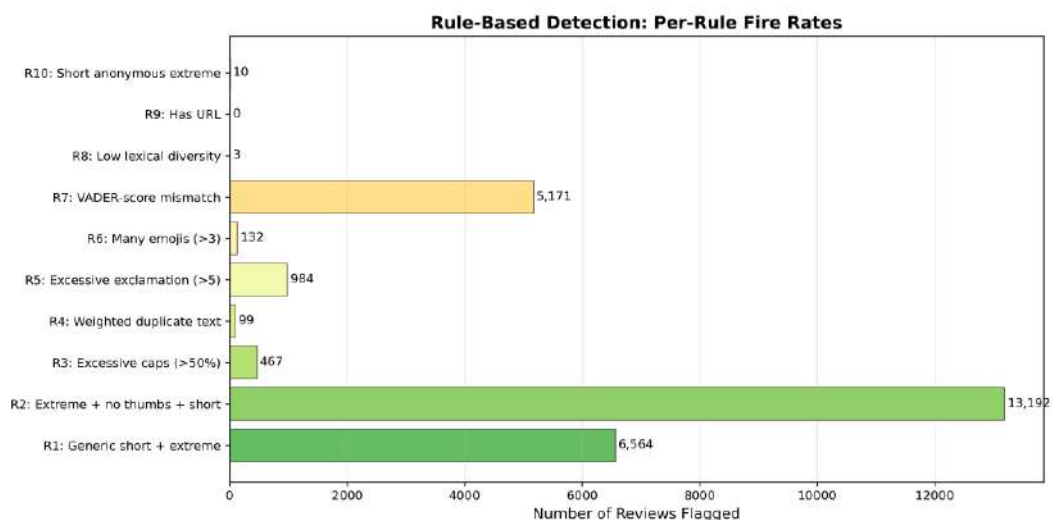


Figure 10: 4.10 Per-rule fire rates

4.2.3 Isolation Forest — Anomaly Detection

Isolation Forest (Liu, Ting, and Zhou, 2008) was selected as the unsupervised anomaly detection algorithm. This method functions by randomly selecting a feature and a split value to recursively partition the data. Anomalous points are isolated with fewer splits, resulting in shorter mean path lengths and lower anomaly scores. The algorithm was selected due to three main advantages: it is tree-based and scale-invariant, eliminating the need for feature normalization; it has $O(n \log n)$ processing complexity and completed training within seconds on 82,069 reviews; and it provides interpretable anomaly scores and feature significance rankings. The model was configured with `n_estimators=200`, increased from the default of 100 to increase stability, `contamination=0.10`, and `random_state=42`. Twenty-one numeric features from Phase 1 served as input. The model identified 8,207 anomalies, representing 10.00% of all reviews.

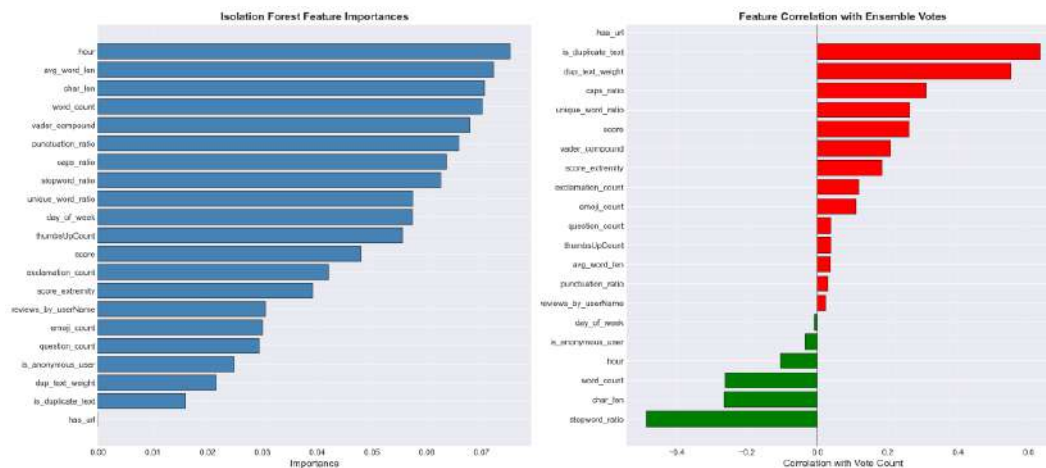


Figure 11: 4.11 Isolation forest feature importance

4.2.4 Contamination Sensitivity Analysis

A sensitivity analysis was performed to justify the contamination parameter selection. Three contamination rates were tested:

Table 3: Table of Contamination sensitivity analysis

Contamination Rate	Anomalies Detected	Percentage
0.05	~4,103	5.00%
0.10	8,207	10.00%
0.15	~12,310	15.00%

The rate of 0.10 was selected as a balanced choice between precision (avoiding false positives) and coverage (catching genuine anomalies). A rate of 0.05 was considered too conservative, while 0.15 risked removing legitimate reviews.

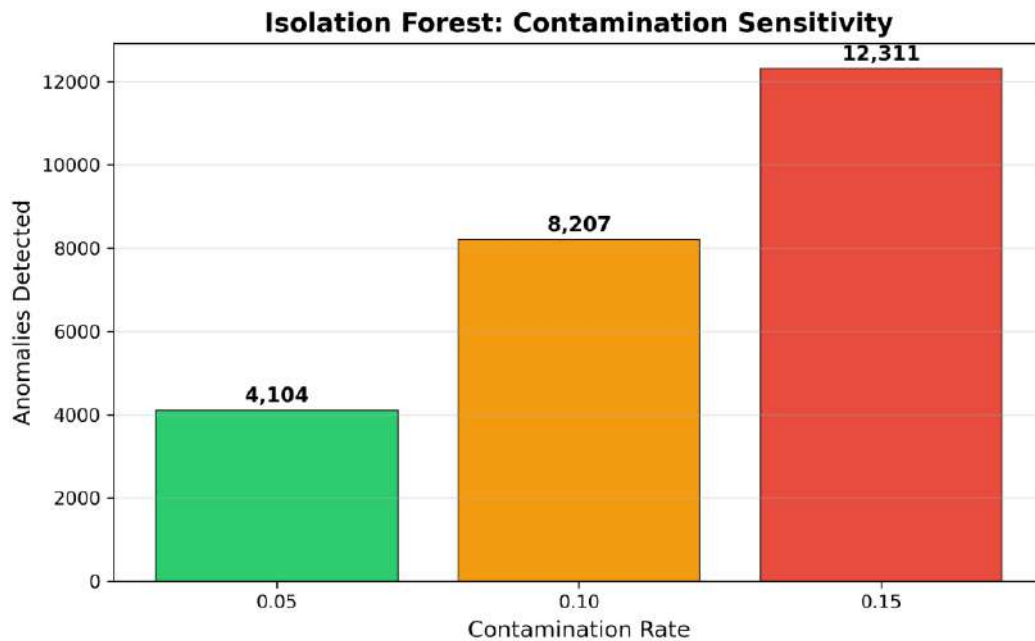


Figure 12: 4.12 Contamination sensitivity analysis

4.2.5 Ensemble Voting & Threshold Selection

All 11 voters (10 rules + Isolation Forest) were combined into an ensemble system. Each review received a total vote count (0–11), and three threshold levels were evaluated:

Table 4: Table of Ensemble voting and threshold selection

Threshold	Votes Required	Reviews Flagged	Percentage
Conservative	≥ 4	50	0.06%
Moderate	≥ 3	2,873	3.50%
Aggressive	≥ 2	8,218	10.01%

The moderate threshold (≥ 3 votes) was selected for the final pipeline which led to the removal of about 2,873 suspicious reviews and the remaining data were used in phase 3. This removed 2,873 suspicious reviews, producing a clean dataset of 79,196 reviews for Phase 3.

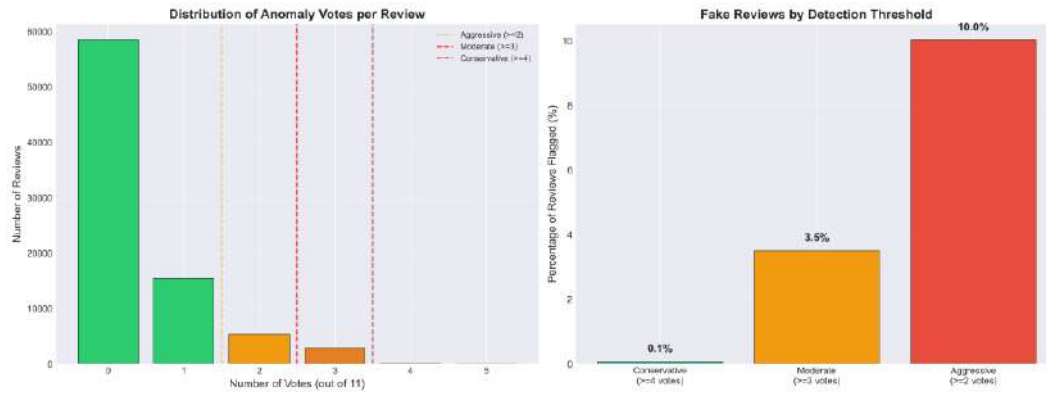


Figure 13: 4.13 Ensemble vote distribution

4.2.6 Validation & Results

The detection results were validated through several analyses. An inter-method agreement heatmap showed the correlation between each voter, confirming that rules and the Isolation Forest captured complementary patterns. A PCA visualisation projected reviews into two dimensions, showing visible separation between flagged and clean clusters. Feature importance analysis revealed which features contributed most to Isolation Forest decisions. A comparison of fake versus real review characteristics confirmed that flagged reviews exhibited shorter lengths, more extreme ratings, and lower lexical diversity than genuine reviews.

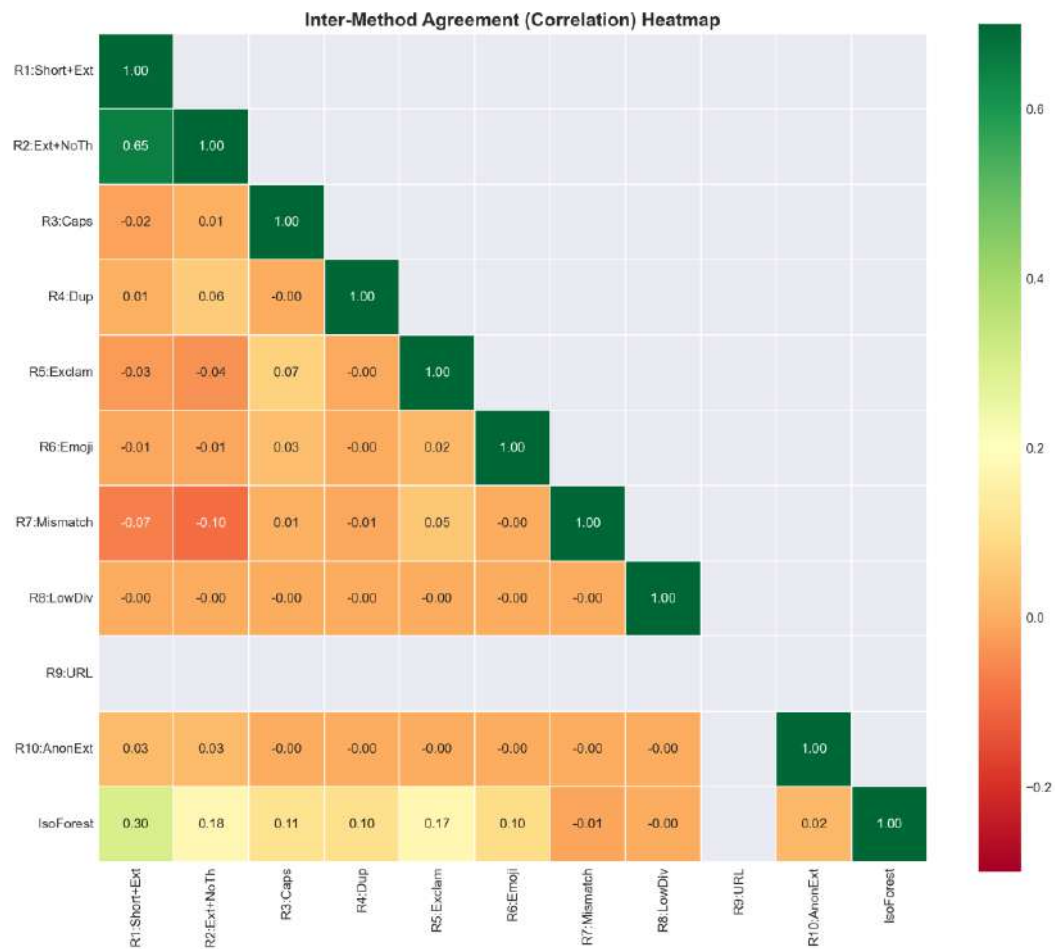


Figure 14: 4.14 Inter-method agreement heatmap

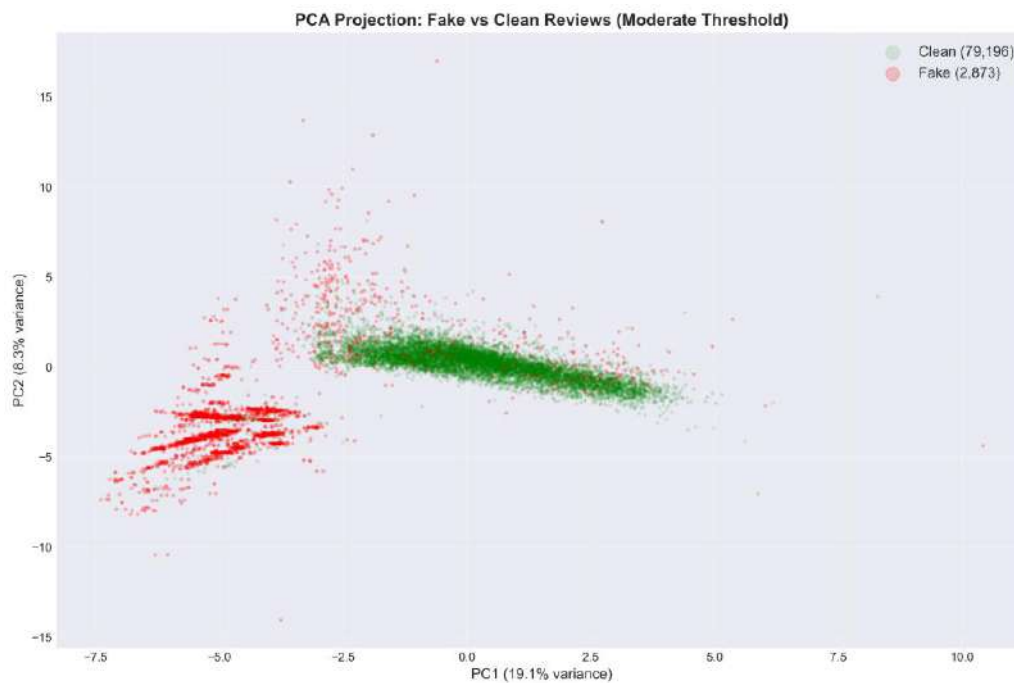


Figure 15: 4.15 PCA - Fake vs Clean Reviews

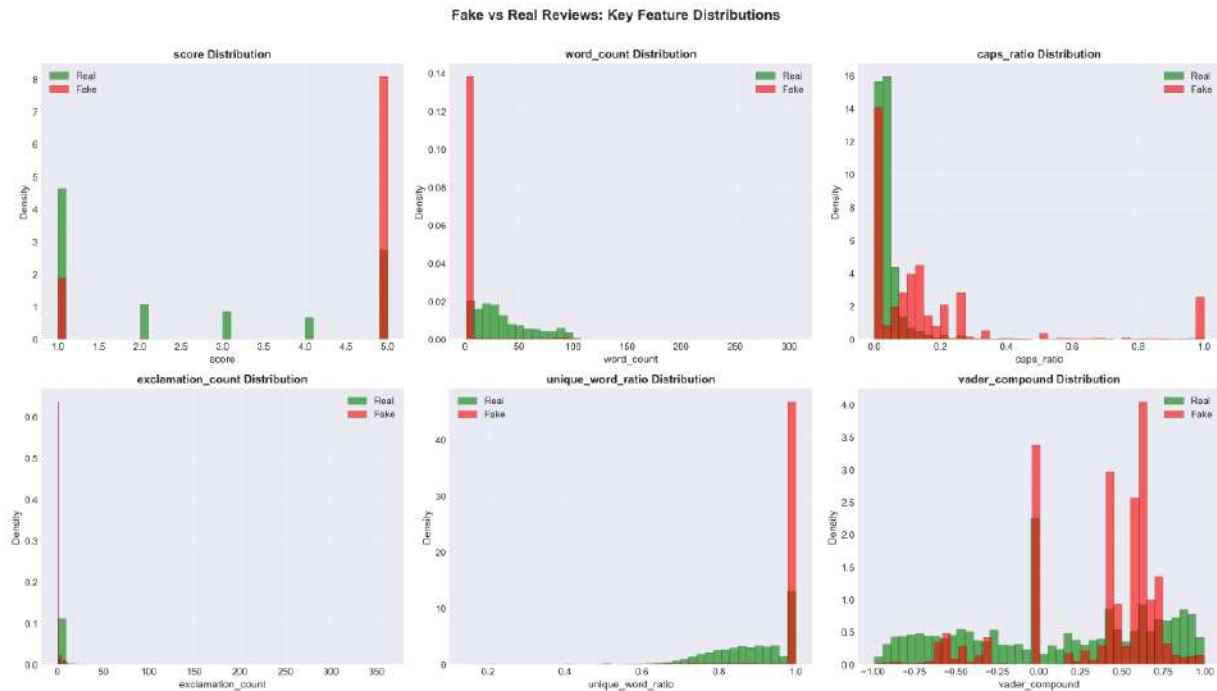


Figure 16: 4.16 Fake vs Real review characteristics

4.3 Phase 3 — Sentiment Analysis

The third phase built and evaluated sentiment classification models on the cleaned dataset (79,196 reviews with fake reviews removed). All work was conducted in Code/[sentimentAnalysis_2models.ipynb](#).

4.3.1 Data Preparation & TF-IDF Vectorisation

Three labels were defined as Negative (1-2 stars), Positive (4-5 stars) and Neutral as (3-star) which was excluded resulting in 72,485 samples (45,379 Negative; 27,106 Positive). 57,988 training and 14,497 test samples were produced by a stratified 80/20 split. Using TF-IDF (max_features=10,000, ngram_range= (1,2)) was used to vectorize the text which enables the model to capture key phrases like “not good” and “very bad”.

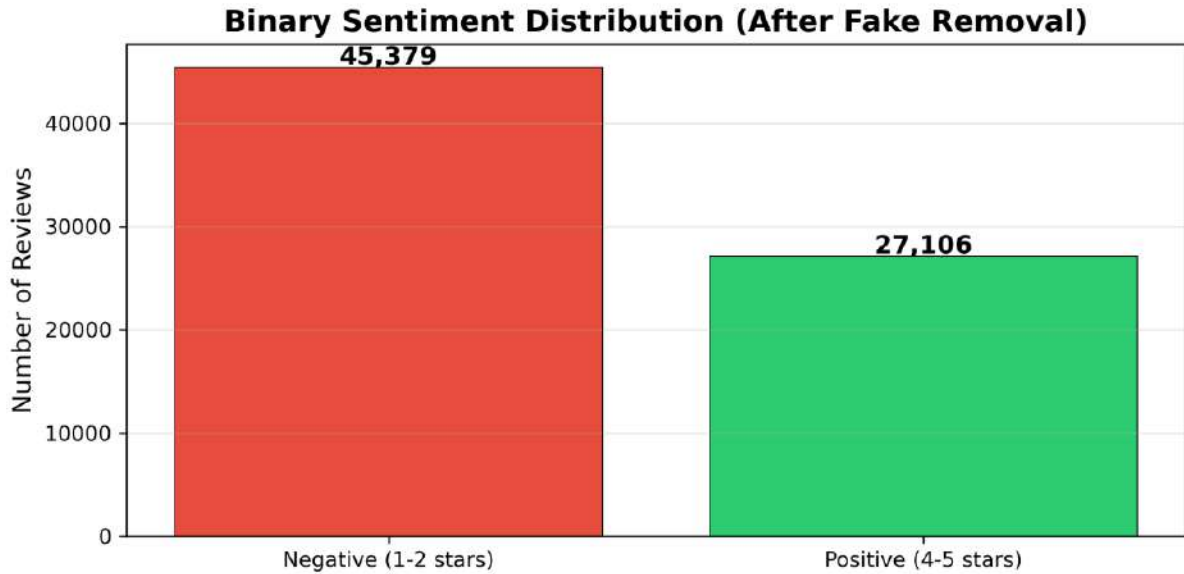


Figure 17: 4.17 Sentiment distribution

4.3.2 Model Training (Logistic Regression & Naive Bayes)

Two classifiers were trained and evaluated:

Logistic Regression: L2-regularised with balanced class weights to handle the class imbalance between negative and positive reviews.

Multinomial Naive Bayes: Configured with Laplace smoothing ($\alpha=0.1$) to handle zero-frequency terms in the TF-IDF matrix.

4.3.3 Model Comparison & Evaluation

Both models were evaluated on the held-out test set and through 5-fold stratified cross-validation. The results are summarized below:

Table 5: Table of Model comparison

Metric	Logistic Regression	Multinomial NB
Accuracy	0.9228	0.9049
F1 (Weighted)	0.9227	0.9027
F1 (Negative)	0.9386	0.9279
F1 (Positive)	0.8961	0.8604
Precision	0.9227	0.9097
Recall	0.9228	0.9049
AUC-ROC	0.9678	0.9561
CV Mean F1 (5-fold)	0.9214 $\pm$ 0.0016	0.9063 $\pm$ 0.0020

Logistic Regression was selected as the best model with F1 score of 0.9227 as it outperformed Multinomial Naive Bayes. The model was validated using confusion matrices and ROC curves with an AUC of 0.9678. The strong generalization was shown by the 5-fold cross validation with a small train test gap of 0.0179.

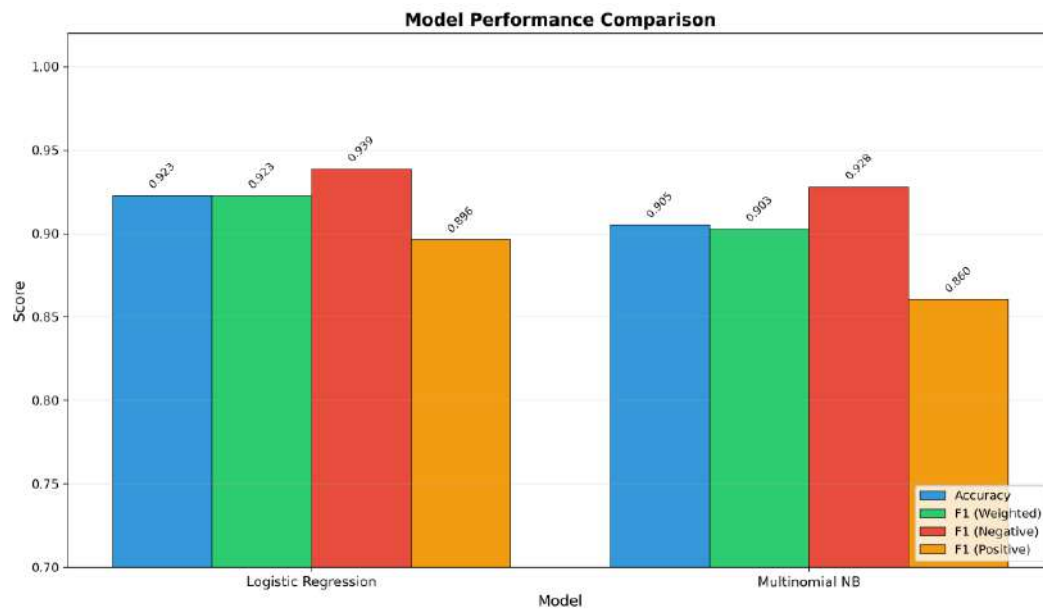


Figure 18: 4.18 Model comparison

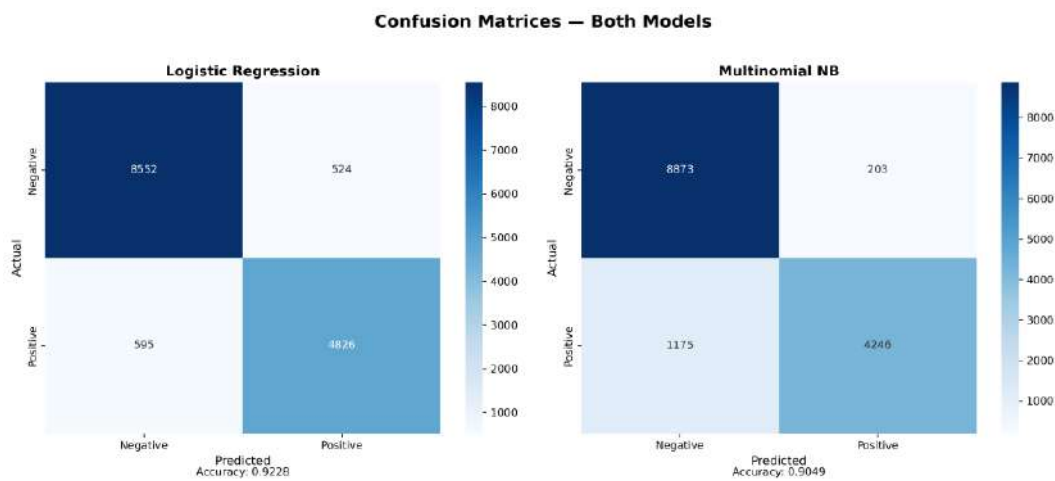


Figure 19: 4.18 Confusion matrices

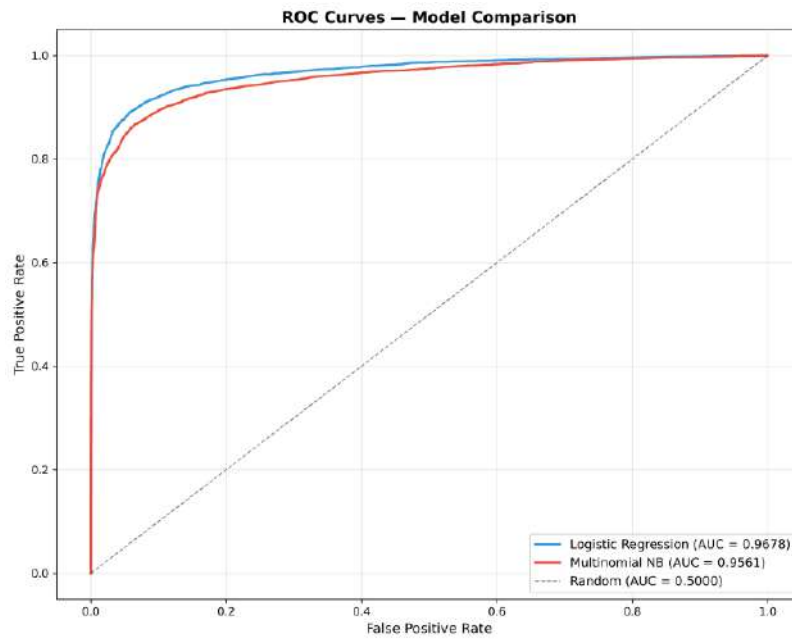


Figure 20: 4.20 ROC Curves

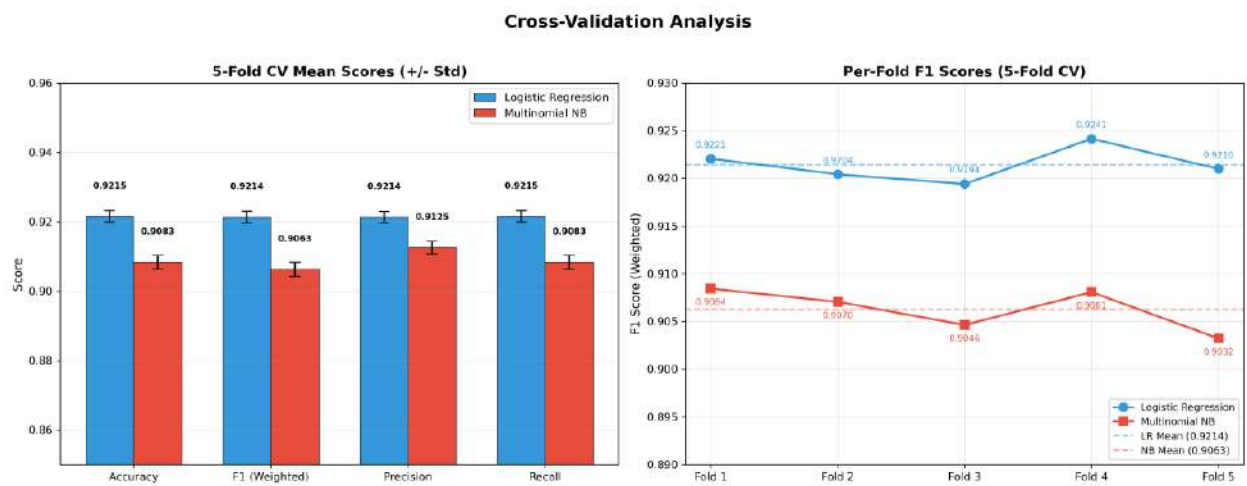
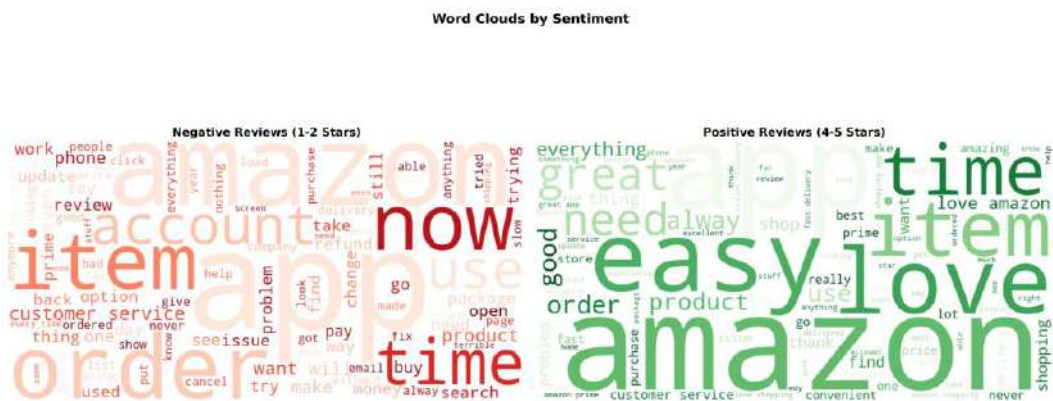
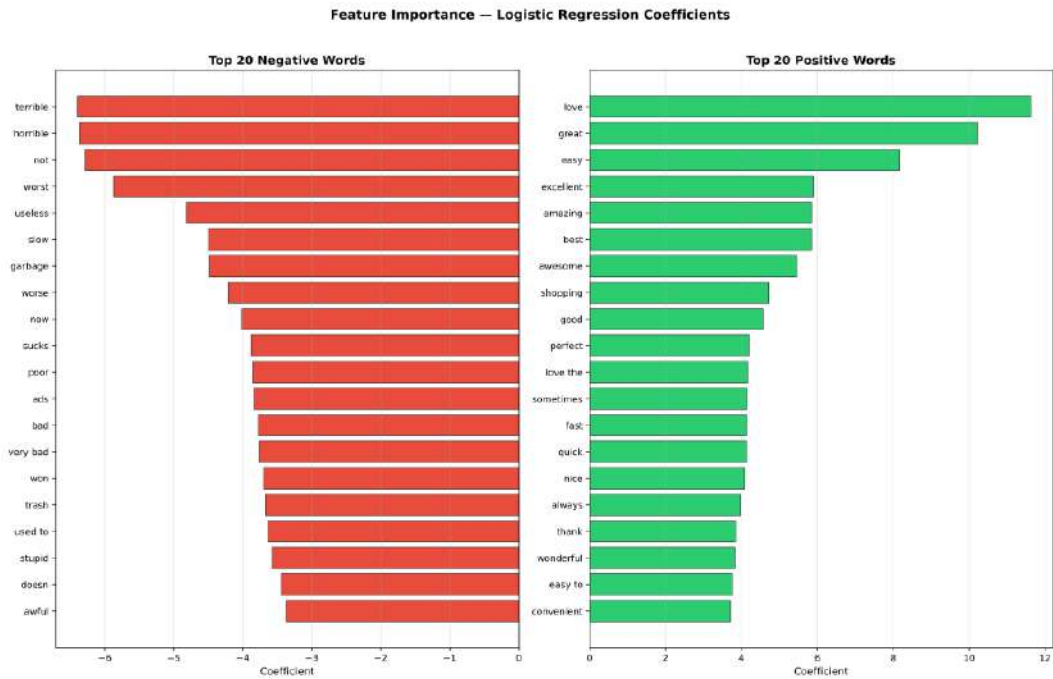


Figure 21: 4.21 5-fold cross-validation results



4.3.4 Impact Analysis — Effect of Fake Review Removal

Two datasets were used to train the logistic regression model in which one was the filtered dataset (79,196 reviews) and another one was the unfiltered dataset (82,069 reviews). The model which was trained on unfiltered data achieved a slightly better F1 score (0.9263 vs. 0.9227, +0.39%), likely because fake reviews are often short and are easier to classify, which inflate the performance. However, the filtered model provides more reliable predictions on genuine reviews, which is the main goal of the pipeline.

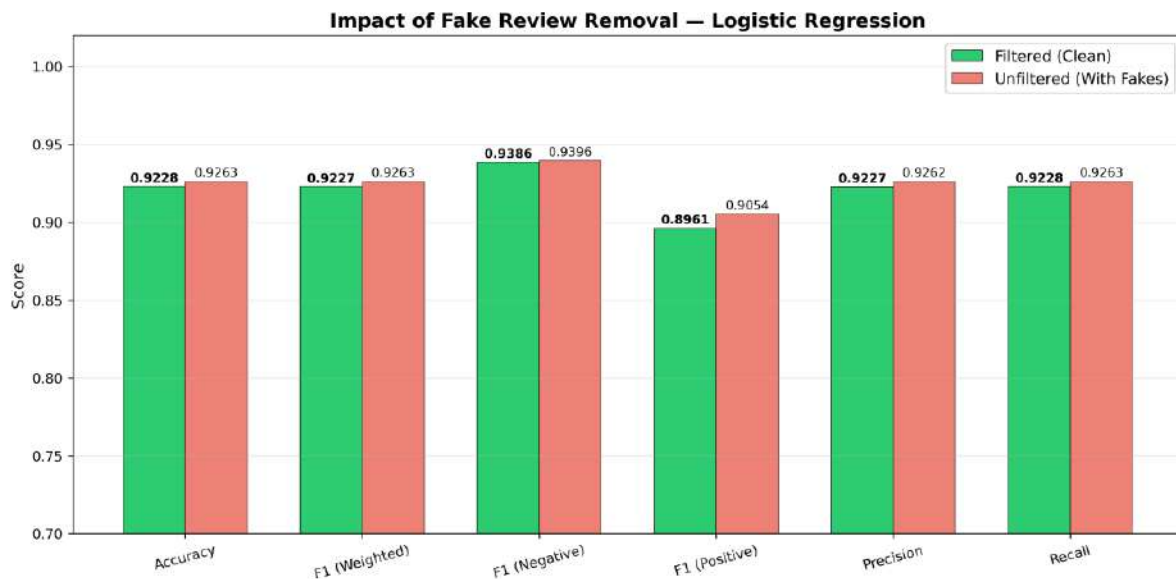


Figure 24: 4.24 Impact of fake reviews removal on sentiment accuracy

4.4 Phase 4 — Interactive Dashboard

A web dashboard was built to ensure even a non-technical user could understand the results easily. The dashboard was built using Streamlit along with Plotly. It combined all the result from earlier phases (data, fake review detection, sentiment results and model performance) into a unified dashboard.

4.4.1 Architecture

The multipage dashboard is designed which caches large dataset to improve the performance. A common stylesheet is being used to ensure consistent design. The model is trained only once at startup, and it is cached to enable fast inference.

4.4.2 Page 1 — Project Overview

The first page is landing page called Project overview which summarizes the pipeline with KPIs tiles showing different information such as total reviews, fake reviews at different threshold such as moderate (2,873), also showing the best model accuracy and many other such important information. It also has some interactive rating distribution chart and a comparison table for both models.

4.4.3 Page 2 — Fake Review Detection

The second page is mainly used to show the phase 2 results. Different tiles are displayed showing the KPIs, rule-based diagnostics, feature comparisons, Isolation Forest insights, PCA visualisation, agreement heatmaps and sensitivity analysis.

4.4.4 Page 3 — Sentiment Analysis

This page is mainly used to display information about the phase 3. Different information is laid out in KPI tiles which summarizes different information. Also, different metrics tables and visualisations are shown such as model comparisons, confusion matrices, ROC curves, cross-validation results, sentiment distribution and key word analysis.

4.4.5 Page 4 — Live Prediction

This page is used to show prediction for the user review. The user can submit a single review or a csv file containing bunch of reviews to predict the sentiment and check if the review is genuine.

The detailed information on Research and Development is available via the project repository ([GitLab Link](#)).

5. Problems Encountered and Solutions:

The detailed information on Problems and Solution is available via the project repository ([GitLab Link](#)).

6. Team Meeting Minute:

Team meeting minutes were recorded throughout the project to capture discussions, decisions, and progress across each phase. This documentation supported effective coordination, transparency, and continuity within the team.

The complete set of meeting minutes can be accessed via the project repository ([GitLab Link](#)).

7. Prototypes, Visualisations and Evaluation:

7.1 Dashboard Prototype:

The final prototype was developed as an analytical dashboard to support the interpretation of Amazon app reviews. The purpose of the dashboard was to combine anomaly detection and sentiment analysis into one clear output so that users could understand both the reliability and emotional tone of those reviews. This directly supports the project aim of improving insights quality in Amazon app reviews.

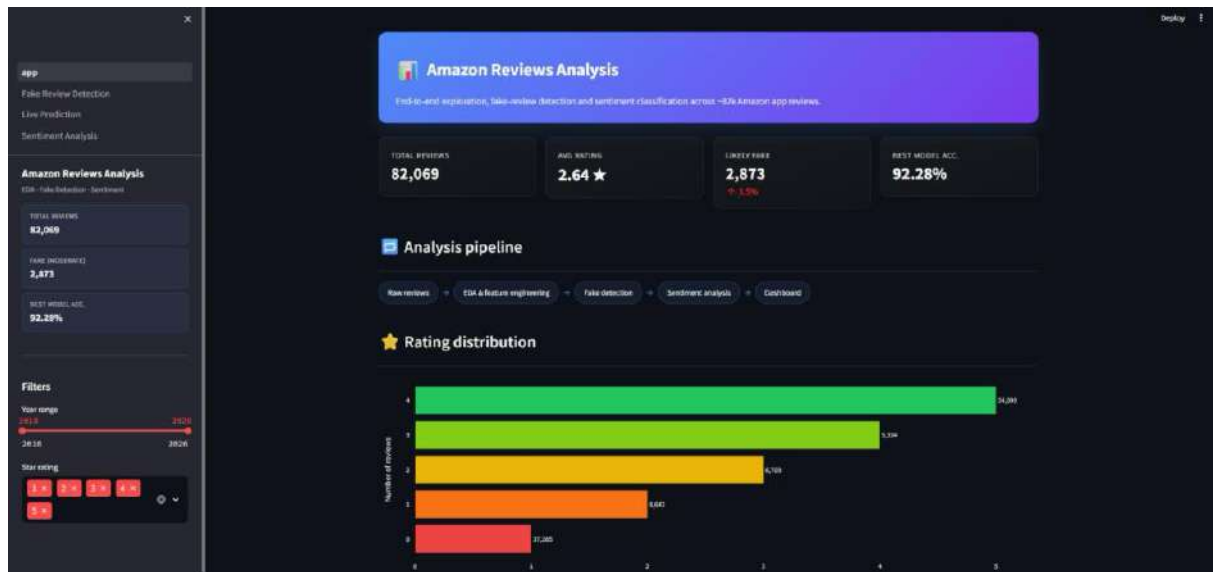


Figure 25: 7.1 Dashboard display

The dashboards show the main output including rating distribution, suspicious review detection, anomaly patterns sentiment classification results, and model evaluation results. The prototype allows the review data to be explore visually rather than only through numerical outputs. This makes the system more useful for decision making because suspicious patterns, review sentiment, and model performance can be understood quickly.

The dashboard demonstrates how the model could be used in practically by allowing the review text to be processed through the fake review detection logic and trained sentiment model.

7.2 Exploratory Data Analysis Output:

The exploratory data analysis (EDA) phase was done to understand the structure, quality, and behavioural patterns present in the Amazon review dataset before applying any modelling techniques. The original dataset consisted of approximately 83,985 reviews. After removing duplicate review IDs and cleaning missing or inconsistent entries, the dataset was reduced to 82,069 valid reviews, ensuring higher data quality for further analysis.

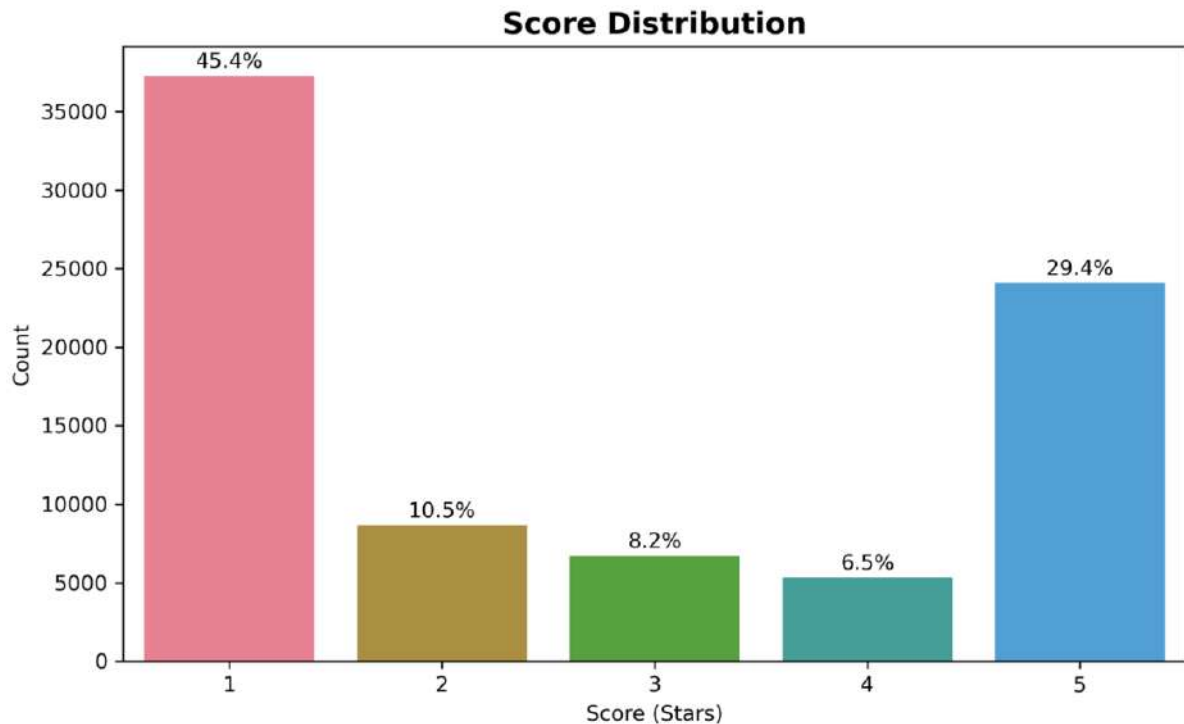


Figure 26: EDA Output - Rating distribution of Amazon app reviews

Several key patterns were identified during this stage. The score-rating distribution showed an uneven spread across star ratings, with a noticeable concentration of higher ratings. For sentiment modelling purposes, 3-star reviews were treated as neutral and excluded, resulting in a sentiment-ready dataset of 75,360 reviews. This dataset contained 45,927 negative reviews (1–2 stars) and 29,433 positive reviews (4–5 stars), indicating a moderate class imbalance.

The analysis of review length and word count revealed that most reviews were relatively short, with only a small proportion of longer, detailed reviews. This pattern suggests that many users provide minimal textual feedback, which can reduce the richness of sentiment information.

To further understand how textual detail relates to user ratings, the relationship between review word count and rating score was analysed.

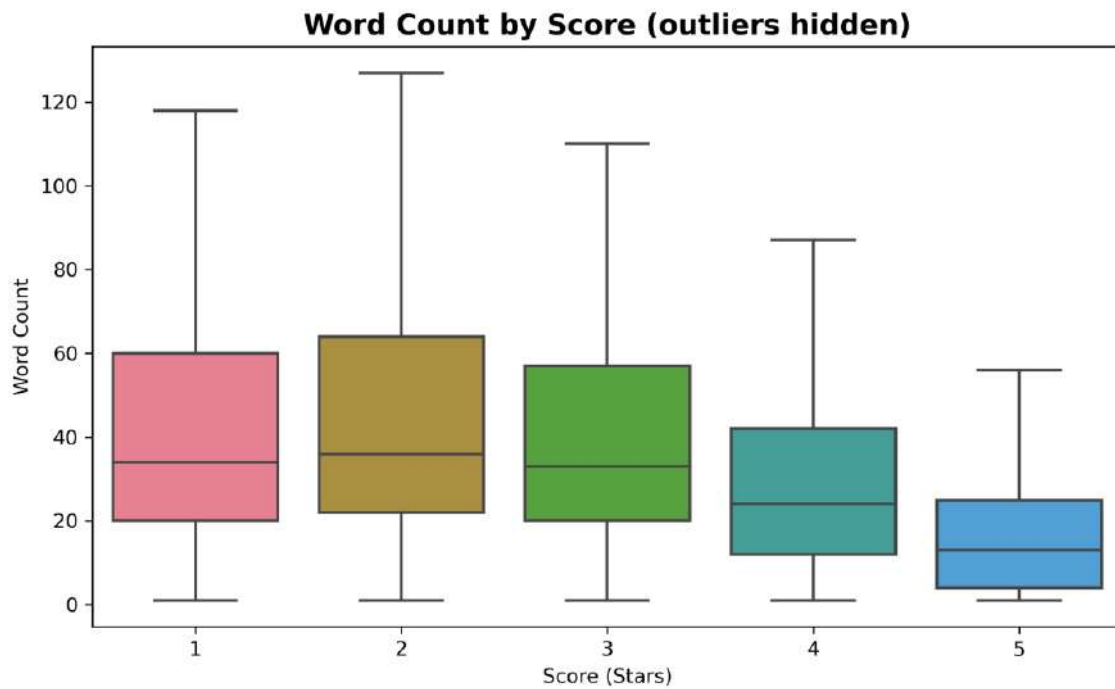


Figure 27: EDA Output - Word count by score

The visualisation shows that shorter reviews are more common across all rating levels, while longer reviews tend to provide more detailed feedback. This suggests that review length alone is not a strong indicator of sentiment polarity, but it remains a useful behavioural feature for anomaly detection.

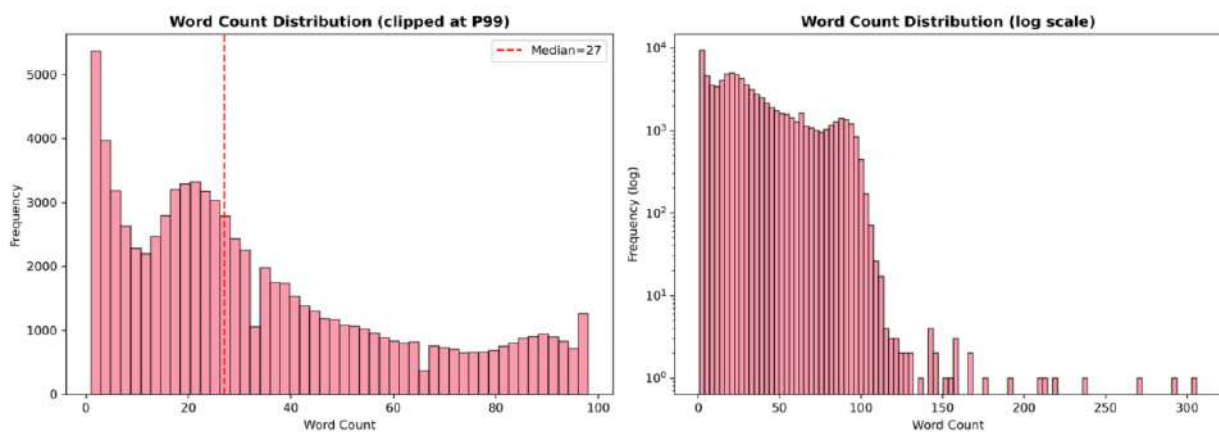


Figure 28: EDA output - Word count distribution

The thumbs-up distribution further supported this observation, with approximately 62% of reviews receiving zero thumbs-up, indicating low user engagement for most reviews.

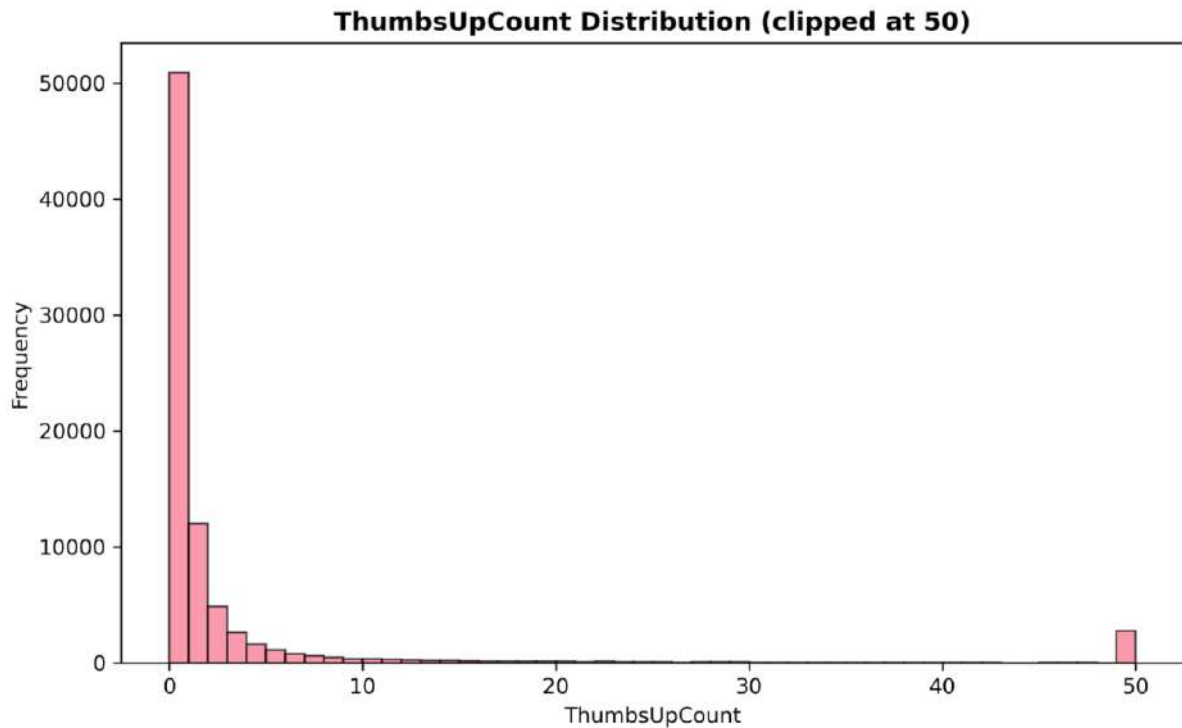


Figure 29: EDA Output - Thumbs-up count distribution

Duplicate text analysis revealed repeated short expressions such as “good”, “great”, and “excellent”, suggesting the presence of low-effort or potentially automated content.

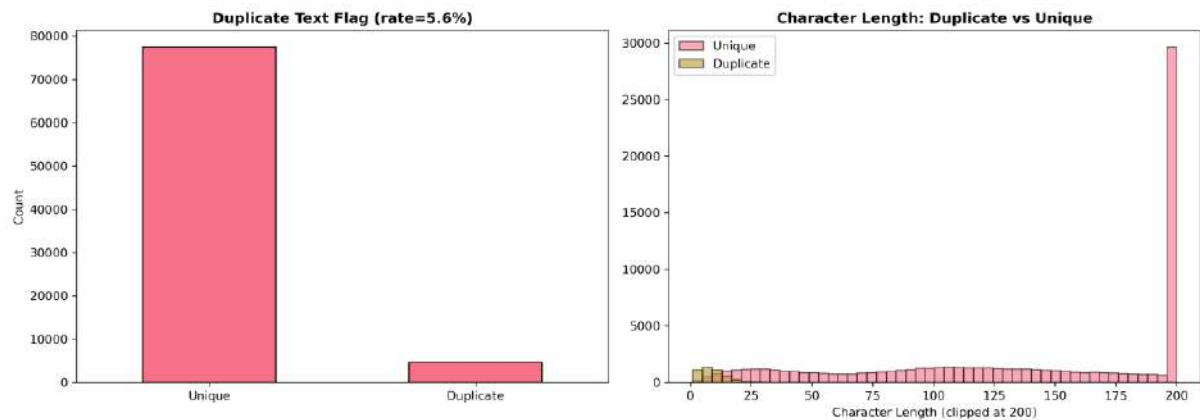


Figure 30: EDA Output - Duplicate text flag and character length

In addition to textual and behavioural features, temporal patterns of review activity were analysed to understand how reviews are distributed over time. The dataset spans from September 2018 to February 2026, with varying levels of review activity across this period. The analysis shows fluctuations in review frequency, including notable peaks where review activity increases significantly on specific dates.

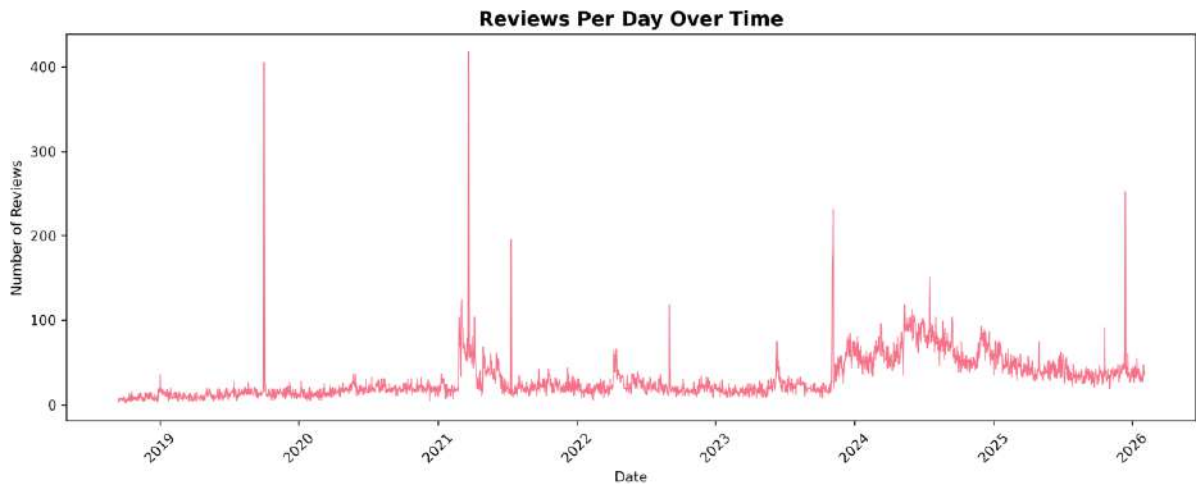


Figure 31: EDA Output - Reviews per day over time

The temporal distribution highlights that review activity is not uniform over time. Certain periods show spikes in the number of reviews submitted, which may indicate events such as product updates, promotional activity, or coordinated review behaviour. These patterns are important for anomaly detection, as unusual spikes in activity can signal potentially suspicious behaviour that requires further investigation.

Additional textual features, including capitalisation ratio, punctuation usage, emoji frequency, and VADER sentiment scores were also examined. These features were later used for anomaly detection and sentiment classification.

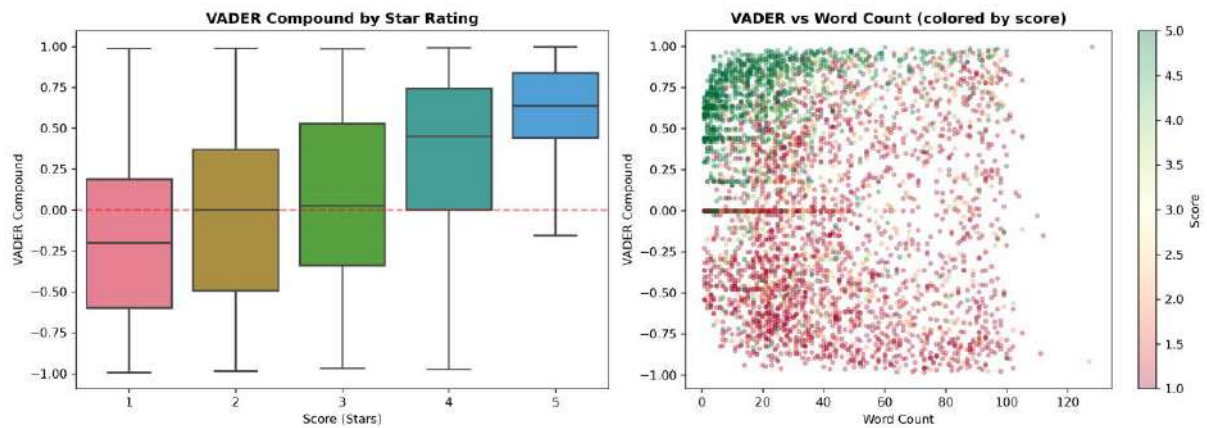


Figure 32: EDA Output - VADER compound by start rating and score

The distribution of reviews across different application versions was also analysed to understand whether user feedback varies with software updates.

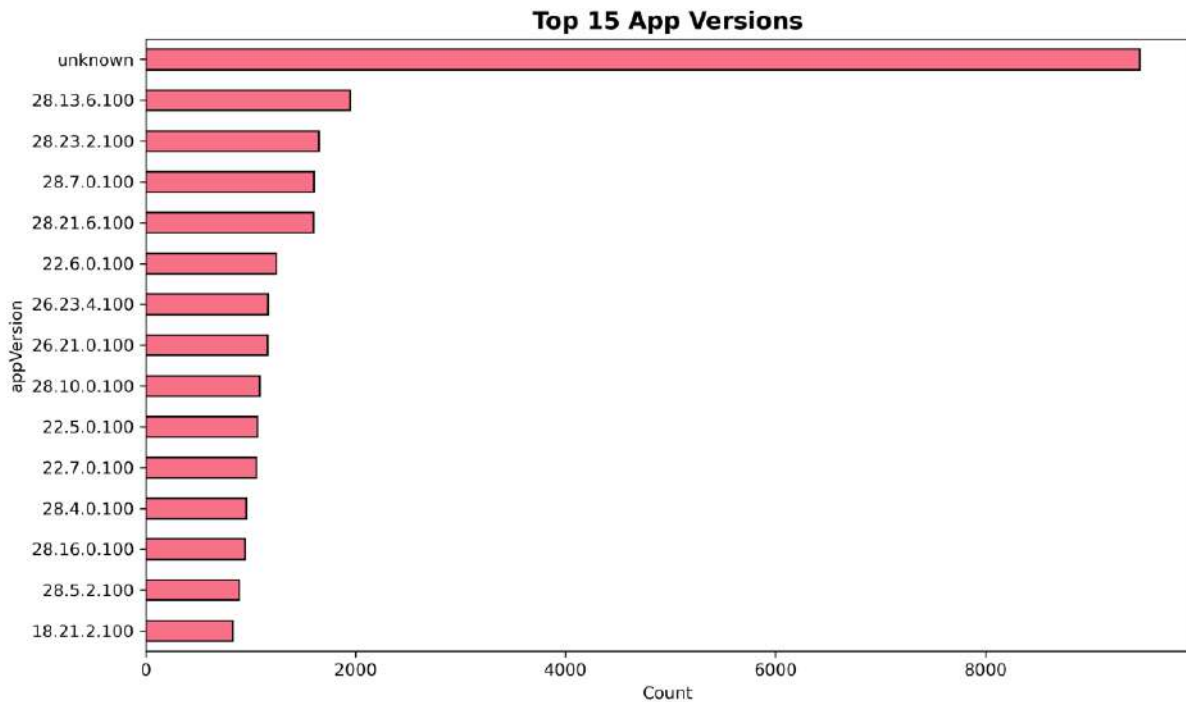


Figure 33: EDA Output - Top 15 app versions

The analysis shows that certain app versions receive significantly more reviews than others. This may reflect periods of increased user activity, updates, or issues affecting specific versions. Such patterns are useful for identifying external factors that influence review behaviour.

Correlation analysis showed strong relationships between certain features, such as character length and word count, as well as moderate relationships between rating scores and sentiment polarity.

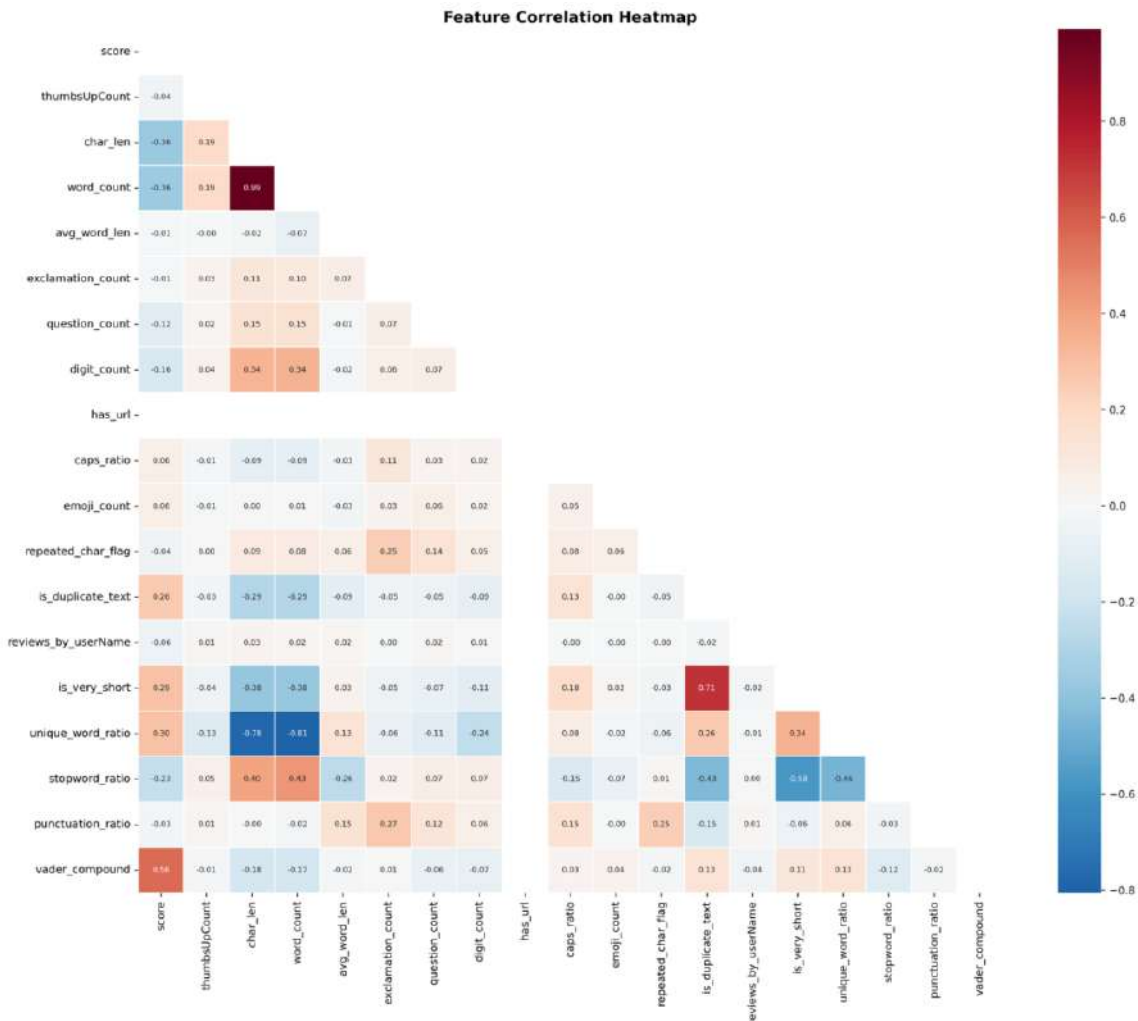


Figure 34: EDA Output - Feature correlation heatmap

These findings confirmed that the selected features captured meaningful behavioural and linguistic patterns within the dataset and provided a strong foundation for subsequent modelling.

Outlier analysis was conducted to identify extreme values in behavioural and textual features that may indicate unusual review patterns.

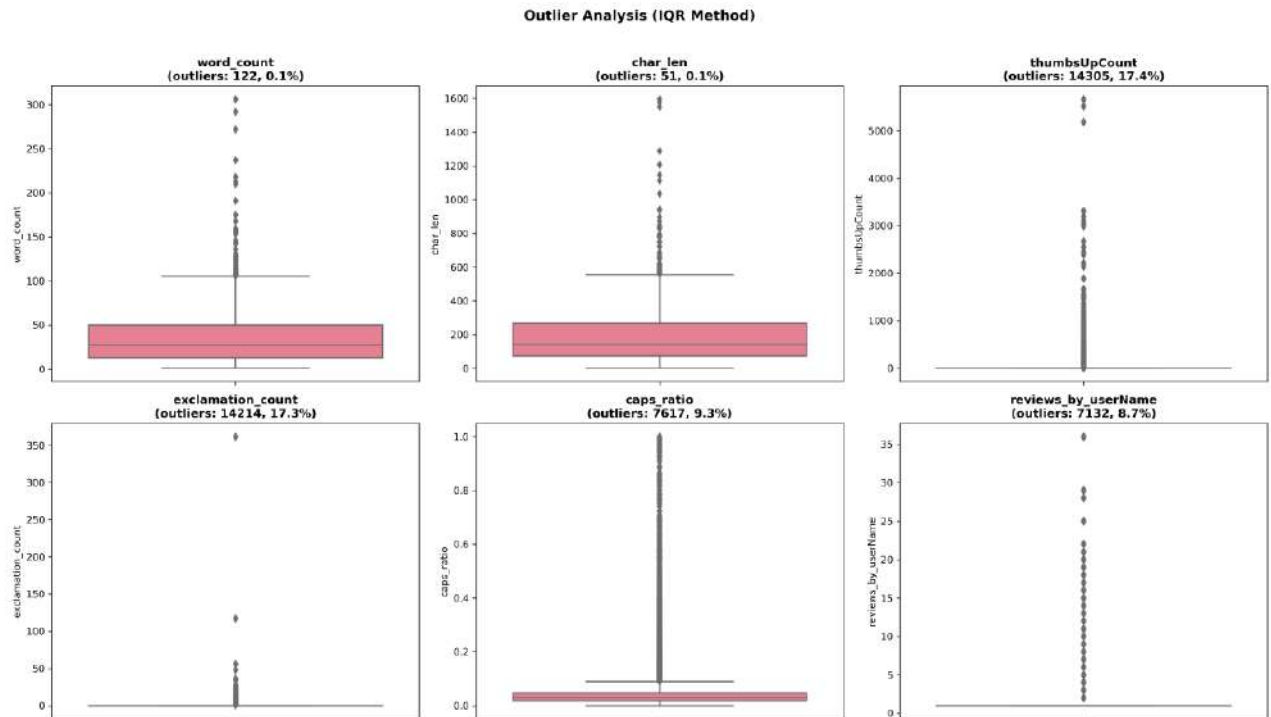


Figure 35: EDA Output (IQR Method)

The presence of outliers in features such as review length, punctuation usage, and rating behaviour suggests that certain reviews deviate significantly from normal patterns. These extreme cases are important inputs for anomaly detection, as they may represent suspicious or non-typical review behaviour.

7.3 Anomaly Detection (Fake Review Detection):

7.3.1 Output and Visualisation:

The anomaly detection aimed to identify suspicious or potentially unreliable reviews using both machine learning and rule-based approaches before proceeding with sentiment analysis.

An Isolation Forest model was applied using 21 engineered features, including review length, average word length, rating behaviour, punctuation ratio, duplicate text weight, and sentiment-related metrics. The model identified 8,207 reviews as anomalous, representing approximately 10% of the dataset.

```

Training Isolation Forest on 82,069 samples with 21 features...

Isolation Forest complete:
  Anomalies detected: 8,207 (10.00%)
  Normal reviews:    73,862
  Anomaly score range: [-0.6787, -0.3582]
  
```

Figure 36: Anomaly Detection Output - IF detection result

This contamination level was selected after testing different thresholds (5%, 10%, and 15%), with 10% providing a balanced detection rate.

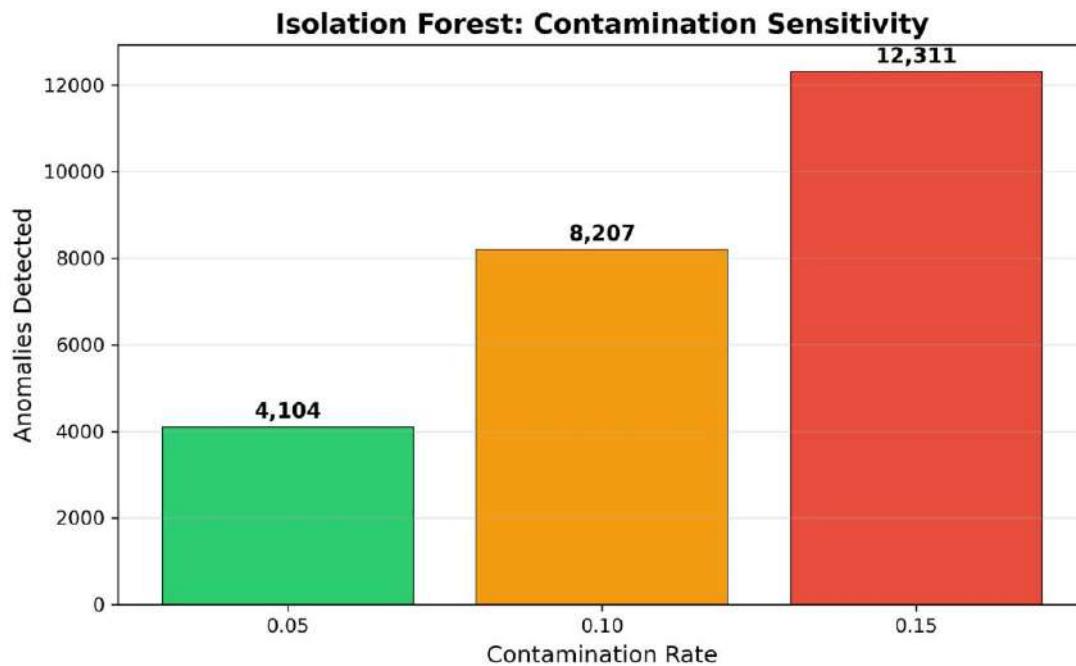


Figure 37: Anomaly Detection Output - Isolation forest contamination sensitivity analysis

In parallel, a rule-based detection system was implemented to capture explicit suspicious patterns. These rules flagged behaviours such as extremely short reviews with extreme ratings, excessive capitalisation, repetitive text, unusual punctuation, emoji overuse, and inconsistencies between rating and sentiment. The rule-based system identified 19,646 reviews that triggered at least one rule.

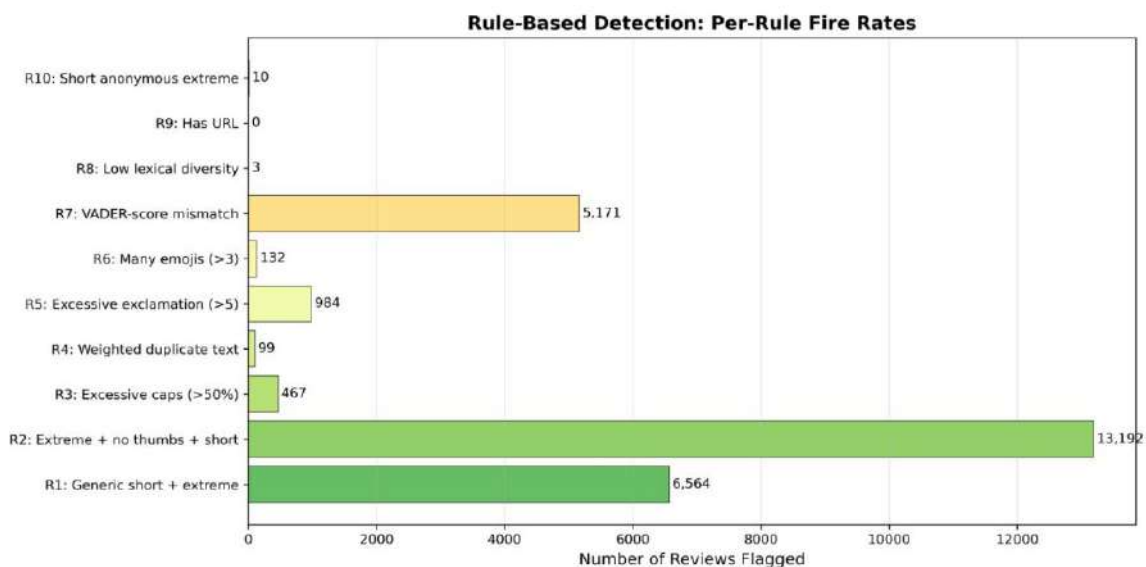


Figure 38: Anomaly Detection Output - Rule-based detection per rule fire rates

To improve reliability, an ensemble voting approach was used. Reviews flagged by three or more rules were considered suspicious, resulting in 2,873 reviews (3.5% of

the dataset). This approach reduced false positives and ensured that only consistently suspicious patterns were highlighted.

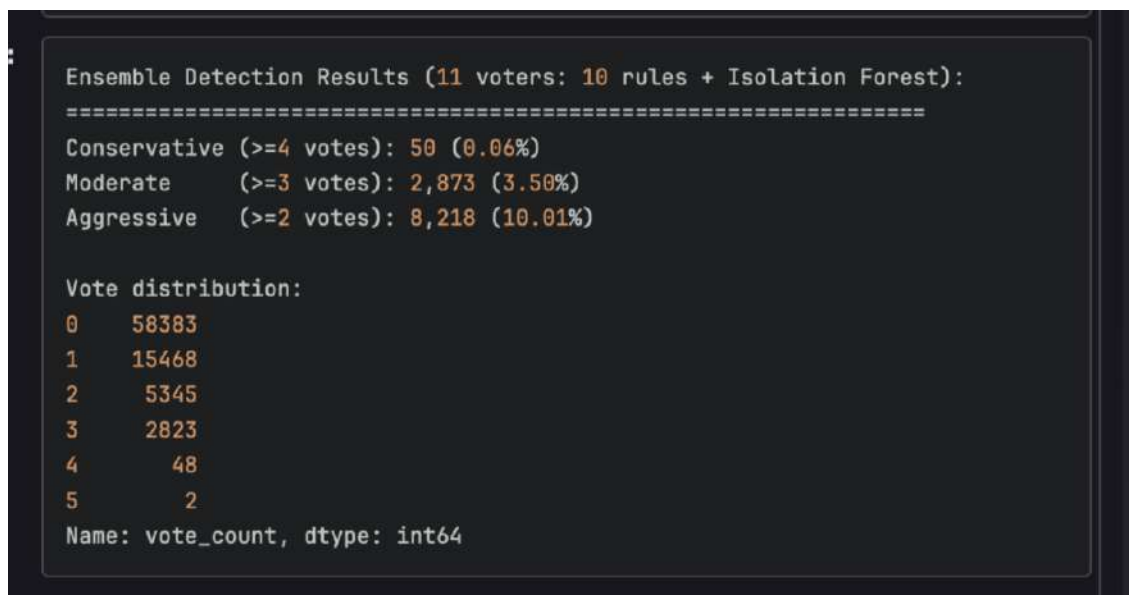


Figure 39: Anomaly Detection Output - Ensemble voting results for suspicious review detection

Several visualisations were used to support interpretation:

Correlation heatmap showing moderate agreement between rule-based and model-based detection methods.

Anomaly score distribution demonstrating that flagged reviews generally had lower anomaly scores.

PCA visualisation reducing feature space to two dimensions and showing partial separation between normal and suspicious reviews.

Feature importance identifying key contributing variables such as behavioural irregularities and exaggerated writing patterns.

These outputs provided both quantitative and visual evidence of suspicious review behaviour. This also shows that linguistic and behavioural patterns affect more than randomness.

7.3.2 Evaluation:

The anomaly detection results demonstrate that suspicious review patterns can be identified using a combination of behavioural and linguistic features. The Isolation Forest model successfully captured unusual review patterns, while the rule-based system highlighted explicit suspicious behaviours. However, each method alone had its own limitations. The Isolation Forest flagged a relatively large proportion of reviews, while the rule-based approach risked over-detection due to simple heuristics.

The ensemble voting approach provided a more balanced solution by combining multiple signals, ensuring that only reviews with consistent suspicious characteristics were flagged. This aligns with existing research which suggests that combining multiple detection techniques improves robustness and reduces redundancy dependence on one method alone (Chandola, Banerjee and Kumar, 2009; Aggarwal, 2017).

The PCA visualisation and anomaly score distribution showed that suspicious reviews do not form a separate cluster. Instead, there is partial overlap between normal and anomalous reviews. This is expected, as fake review detection is an unsupervised problem without confirmed ground-truth labels for direct accuracy testing (Liu, Ting and Zhou, 2008; Emmott et al., 2013). As a result, anomaly detection should be interpreted as identifying suspicious patterns rather than definitive fake reviews.

Overall, the anomaly detection system provides a meaningful way to assess review reliability. While it cannot guarantee absolute accuracy, it effectively highlights patterns that may distort customer perception and supports more informed interpretation of review data.

7.4 Sentiment Analysis:

7.4.1 7.4.1 Output and Visualisation:

The sentiment analysis focused on classifying reviews into positive and negative categories based on textual content. After preprocessing, the sentiment-ready dataset contained 75,360 reviews, with 45,927 negative and 29,433 positive instances.

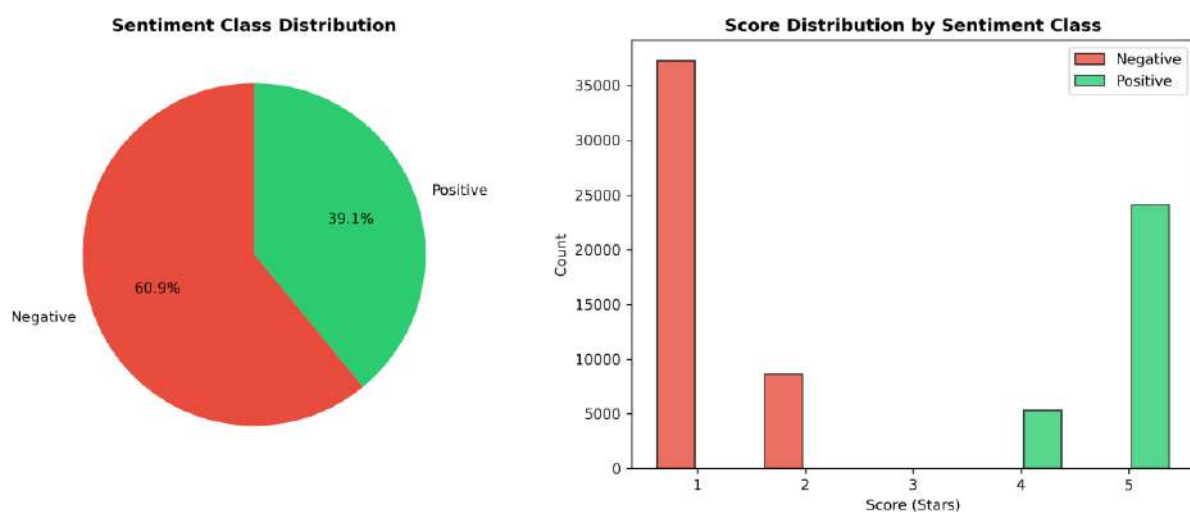


Figure 40: Sentiment Analysis - Output: Sentiment class distribution after neutral review removal

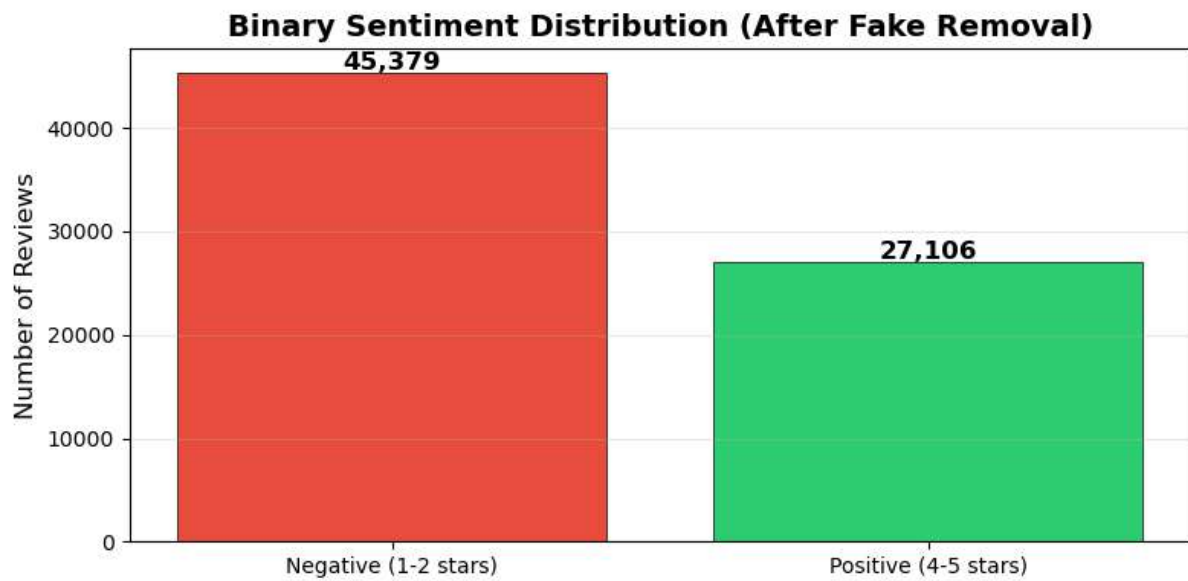


Figure 41: Sentiment Analysis - Output: Binary sentiment distribution after fake removal

Two machine learning models were evaluated: Logistic Regression and Multinomial Naive Bayes. Both models used TF-IDF vectorisation with unigrams and bigrams, limited to 10,000 features.

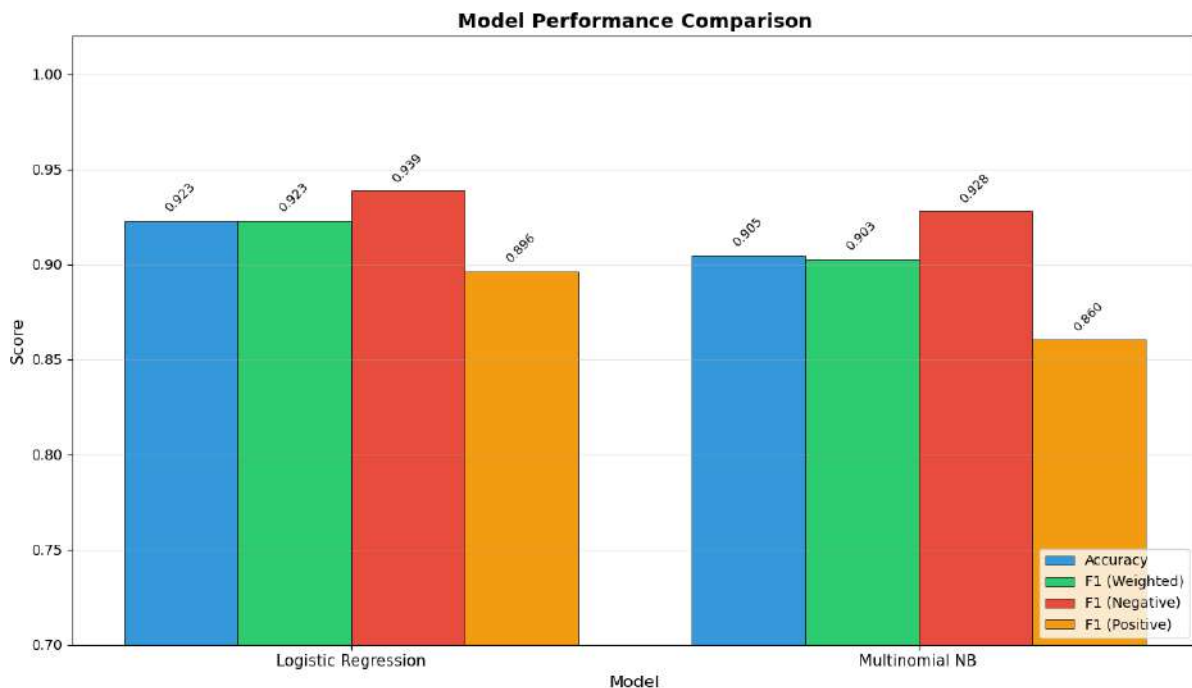


Figure 42: Sentiment Analysis - Output: Comparing model performance

The results showed that Logistic Regression achieved superior performance, with:
Accuracy: 92.28%

Weighted F1-score: 0.9227

ROC AUC: 0.9678

In comparison, Multinomial Naive Bayes achieved:

Accuracy: 90.49%

Weighted F1-score: 0.9027

ROC AUC: 0.9561

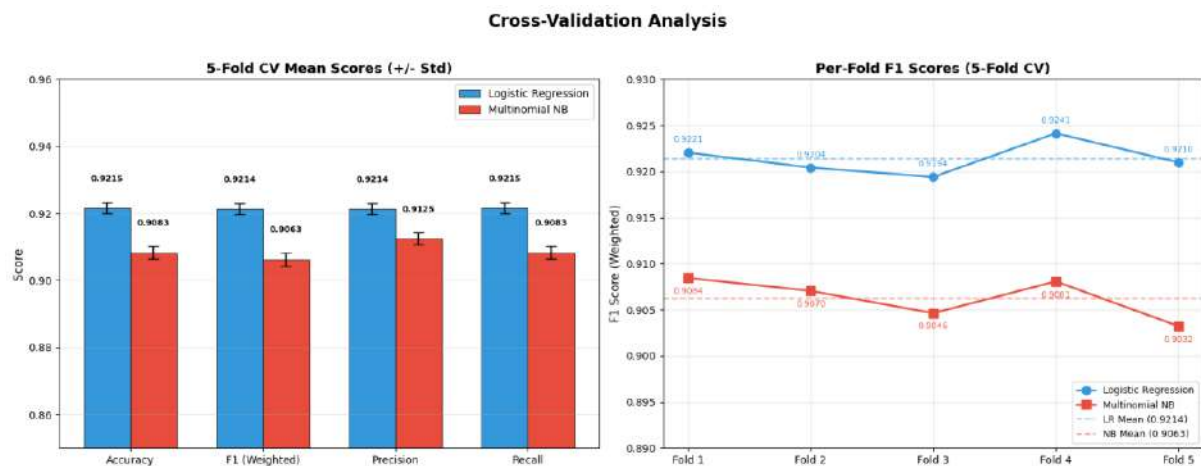


Figure 43: Sentiment Analysis - Output: 5-fold cross validation analysis

To ensure robustness, stratified 5-fold cross-validation was applied. The Logistic Regression model achieved an average weighted F1-score of 0.9214 ± 0.0016 , indicating very low variance across folds and strong generalisation performance.

Several visualisations were used to interpret the results:

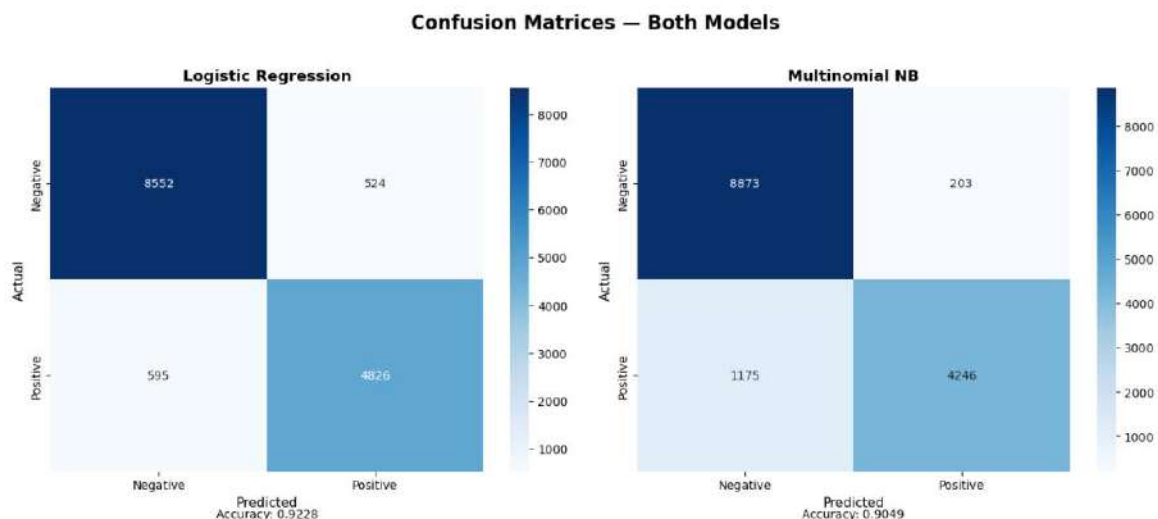


Figure 44matrices for Logistic Regression and Multinomial Naive Bayes: Sentiment Analysis - Output: Confusion

Confusion matrix, showing correct and incorrect classifications

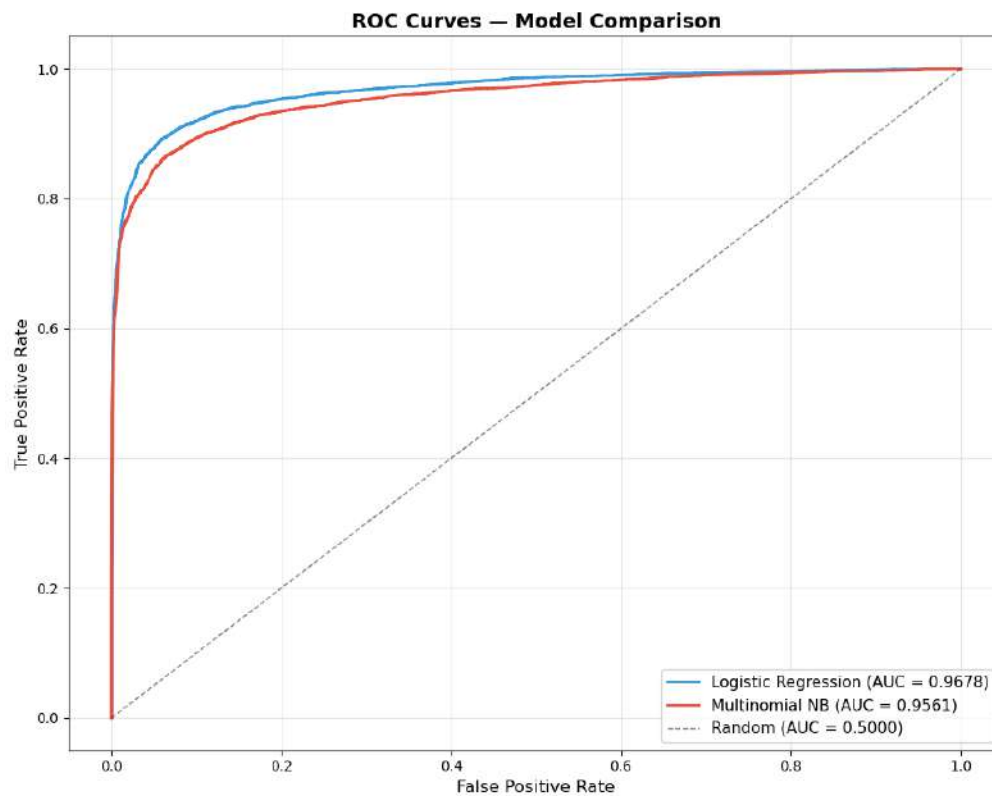


Figure 45: Sentiment Analysis Output: ROC curved for sentiment classification models

ROC curve, demonstrating strong class separability



Figure 46: Sentiment Analysis Output: Word cloud for positive and negative reviews

Word clouds, highlighting dominant terms in positive and negative reviews

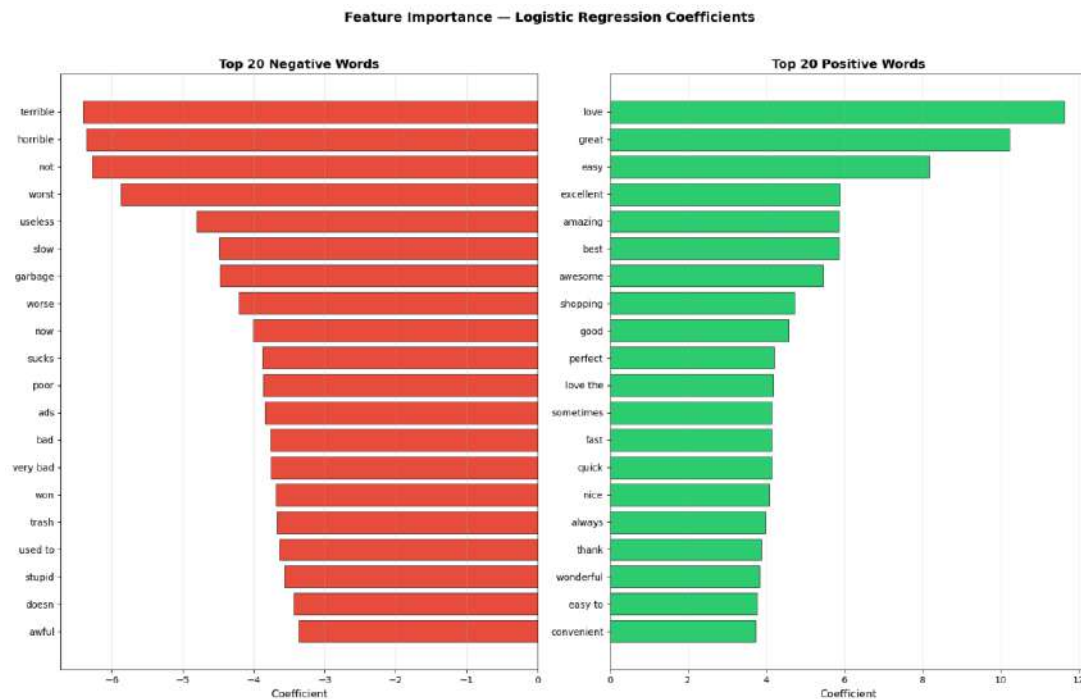


Figure 47: Sentiment Analysis Output: Feature importance on top 20 positive and negative words

Feature importance, identifying key words influencing classification decisions

These outputs confirmed that the model captured meaningful linguistic patterns.

7.4.2 7.4.2 Evaluation:

The sentiment analysis results indicate that Logistic Regression is the most suitable model for this dataset. It consistently outperformed Multinomial Naive Bayes across all evaluation metrics. The high ROC, AUC value (0.9678) demonstrates strong discrimination between positive and negative classes, indicating that the model is effective at capturing sentiment patterns.

The use of cross-validation further strengthens the reliability of the results, as it confirms that the model's performance is stable and not dependent on a single data split. This is particularly important given the moderate class imbalance in the dataset, as stratified validation helps produce more reliable evaluation results (Kohavi, 1995).

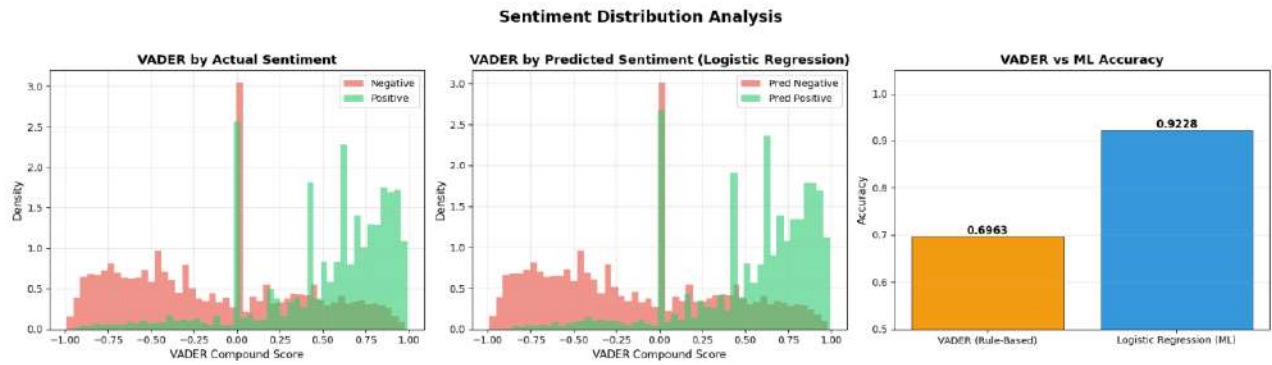


Figure 48: Sentiment distribution analysis

The VADER comparison further shows why the final sentiment classification relied on machine learning rather than rule-based sentiment scoring alone. Although VADER was useful for exploratory analysis and feature engineering, its accuracy was lower than Logistic Regression. This suggests that the TF-IDF-based Logistic Regression model captured review language more effectively and produced stronger sentiment classification results.

The visualisation outputs also support model validity. The word clouds and feature importance analysis show that the model relies on intuitive and interpretable language patterns, such as positive expressions (“love”, “great”, “excellent”) and negative sentiment indicators. This aligns with established sentiment analysis research, which emphasises the importance of checking whether models learn meaningful sentiment-related language patterns rather than relying only on accuracy scores (Pang, Lee and Vaithyanathan, 2002).

Overall, the sentiment analysis component performs effectively and provides reliable classification of review polarity, making it suitable for understanding customer opinions at scale.

7.5 Integrated System Evaluation:

The combined results from EDA, Anomaly Detection, and Sentiment Analysis demonstrate that the project successfully developed a structured analytical pipeline for improving insight quality in Amazon app reviews.

The EDA stage provided important insights into data quality and user behaviour, highlighting issues such as duplicate reviews, low engagement, and short-text patterns. These findings informed feature engineering and model design.

The anomaly detection system added a layer of reliability assessment by identifying suspicious reviews using both machine learning and rule-based approaches. This is important because previous fake review research shows that deceptive or suspicious reviews often contain behavioural and linguistic signals that can affect the reliability of review-based insights (Jindal and Liu, 2008; Mukherjee et al., 2013). While the absence of ground-truth labels limits definitive validation, the use of multiple evaluation

techniques including ensemble voting, PCA visualisation, and score distribution analysis, provides strong supporting evidence.

The sentiment analysis model demonstrated high performance and stability, with Logistic Regression achieving strong classification accuracy and interpretability. The results confirm that meaningful sentiment patterns can be extracted from user-generated review data.

Importantly, the integration of anomaly detection and sentiment analysis supports the overall project aim. By identifying suspicious reviews before interpreting sentiment, the system helps ensure that insights are derived from more reliable data.

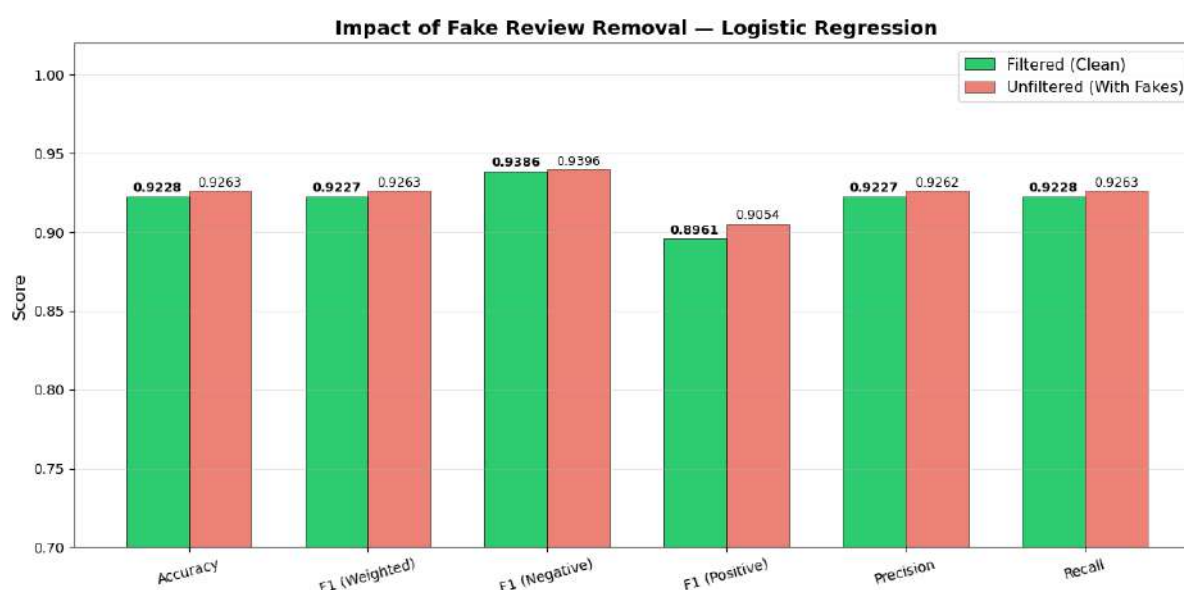


Figure 49: Evaluation: Impact of fake review removal on sentiment classification

However, the results also show that removing suspicious reviews does not necessarily improve classification accuracy, highlighting the distinction between data reliability and model performance. The comparison shows that removing suspicious reviews did not significantly improve the Logistic Regression model's classification performance. In fact, the unfiltered data set performed slightly higher across some metrics. This suggests that the anomaly detection is more valuable for improving the reliability and trustworthiness of review interpretation rather than directly increasing sentiment classification accuracy.

In conclusion, the project provides a well-supported analytical framework that combines multiple techniques to enhance the interpretation of online reviews. While limitations remain due to the unsupervised nature of fake review detection, the approach demonstrates both technical effectiveness and critical evaluation, making it suitable as a data artefact for real-world applications.

The detailed information on Prototype, Evaluation, and Output is available via the project repository ([GitLab Link](#)).

8. Sprint Retrospectives:

Table 6: Table of Sprint retrospectives

Sprint	Focus	What Went Well	Problems	Improvement
Sprint 1	Topic, research, dataset	Clear project direction	Dataset uncertainty	Confirm dataset earlier
Sprint 2	Data cleaning and EDA	Useful patterns found	Noisy data	Improve preprocessing
Sprint 3	Anomaly detection	Isolation Forest implemented	No true labels	Use score/PCA validation
Sprint 4	Sentiment model	Model comparison completed	Rating labels imperfect	Explain limitation
Sprint 5	Dashboard	Prototype created	Integration issues	Test with sample inputs
Sprint 6	Final documentation	Report organised	Missing evidence	Add appendices/screenshots

It is also available via the repository ([GitLab Link](#)).

9. Code and Technical Artefacts:

9.1 Link to repositories:

The Link to the repository for this whole with code and artefact is available via the repository; the link is here - ([GitLab Link](#)).

10. Reflection:

10.1 Group Reflection:

As a group, this project helped us understand how a data science project develops from an initial idea into a working final artefact. At first, we focused mainly on sentiment analysis, but during development we realised that review reliability was also important. This led us to combine anomaly detection with sentiment analysis so that suspicious review patterns could be considered before interpreting customer opinions.

The team worked in different areas, including data cleaning, EDA, anomaly detection, sentiment modelling, dashboard development, literature review, and documentation. We faced challenges with dataset cleaning, model evaluation, dashboard integration, and time management, but we improved our process by using Trello, GitLab, meeting minutes, and clearer task allocation. This made the project more organised and helped everyone understand their responsibilities.

Overall, the project improved our technical and teamwork skills. We learned that real-world data projects require flexibility because problems such as missing labels, noisy data, and integration issues cannot always be fully predicted at the start. Although fake review detection could not be proven with confirmed labels, the team handled this limitation honestly using suitable evaluation methods, making the outcome more reliable and well-justified. This also helped us become more confident as a team because we learned how to adjust our plan, communicate, and manage instead of getting stuck when something did not work as expected.

10.2 Individual Reflection (Pranika):

In this project, I worked as the team leader, where I coordinated the team, arranged meetings, recorded meeting minutes, allocated tasks, and supported the final report writing. I also contributed technically to the anomaly detection evaluation, outcome interpretation, and the sentiment dashboard page. At the beginning, team coordination was sometimes challenging because some members agreed with ideas without sharing many of their own opinions, so I learned that leadership is not only about directing tasks but also encouraging discussion and involvement.

This project helped me improve my confidence, communication, teamwork, and technical understanding of machine learning models. Working on anomaly evaluation,

sentiment analysis, and Streamlit also helped me develop new technical skills. Overall, I learned how to manage a team, adapt when problems appear, and improve collaboration through clearer planning and communication. In future projects, I would encourage more active discussion by asking open questions and giving members chances to lead smaller parts of the work.

11. Conclusion:

This project successfully developed an analytical framework for improving insight quality in Amazon app reviews by combining exploratory data analysis, anomaly detection, sentiment analysis, and dashboard visualisation. Through data cleaning and EDA, the team identified important patterns such as duplicate reviews, short review texts, class imbalance, user engagement differences, and unusual review behaviour. These findings helped guide the feature engineering, anomaly detection, and sentiment modelling stages.

The anomaly detection component used Isolation Forest, rule-based indicators, and ensemble voting to identify suspicious review patterns. Since confirmed fake review labels were not available, the system was designed to flag potentially unreliable reviews rather than prove that a review was fake. The sentiment analysis component compared Logistic Regression and Multinomial Naive Bayes, with Logistic Regression performing best across accuracy, F1-score, and ROC AUC. Together, these results showed that combining review reliability checks with sentiment classification provides a more meaningful way to interpret customer feedback.

Overall, the project met its main objectives by producing a working data science pipeline, clear evaluation outputs, and a dashboard prototype. The team also developed stronger skills in data preprocessing, machine learning, evaluation, project management, and collaboration. Although limitations remain, particularly the lack of labelled fake review data, the final outcome provides a useful and well-supported approach for analysing online review quality and sentiment.

12. References:

Aggarwal, C.C. (2017) *Outlier Analysis*. 2nd edn. Cham: Springer.

Chandola, V., Banerjee, A. and Kumar, V. (2009) 'Anomaly detection: A survey', *ACM Computing Surveys*, 41(3), pp. 1–58.

Du, J., Rong, J., Michalska, S. and Wang, H. (2019) Feature Selection For Helpfulness Prediction of Online Product Reviews. an Empirical Study. [online]. [Accessed 01 December 2025].

Emmott, A., Das, S., Dietterich, T., Fern, A. and Wong, W. (2013) 'A meta-analysis of the anomaly detection problem', *Proceedings of the 2013 SIAM International Conference on Data Mining*, pp. 920–928.

Hölzing, C., Meybohm, P. and Happel, O. (2025) Transformer Models Enhance Explainable Risk Categorization of Incidents Compared to Tf-idf Baselines. [online]. [Accessed 01 December 2025].

Jindal, N. and Liu, B. (2008) Opinion Spam and Analysis. [online]. [Accessed 01 December 2025].

Kohavi, R. (1995) 'A study of cross-validation and bootstrap for accuracy estimation and model selection', *Proceedings of the 14th International Joint Conference on Artificial Intelligence*, 2, pp. 1137–1143.

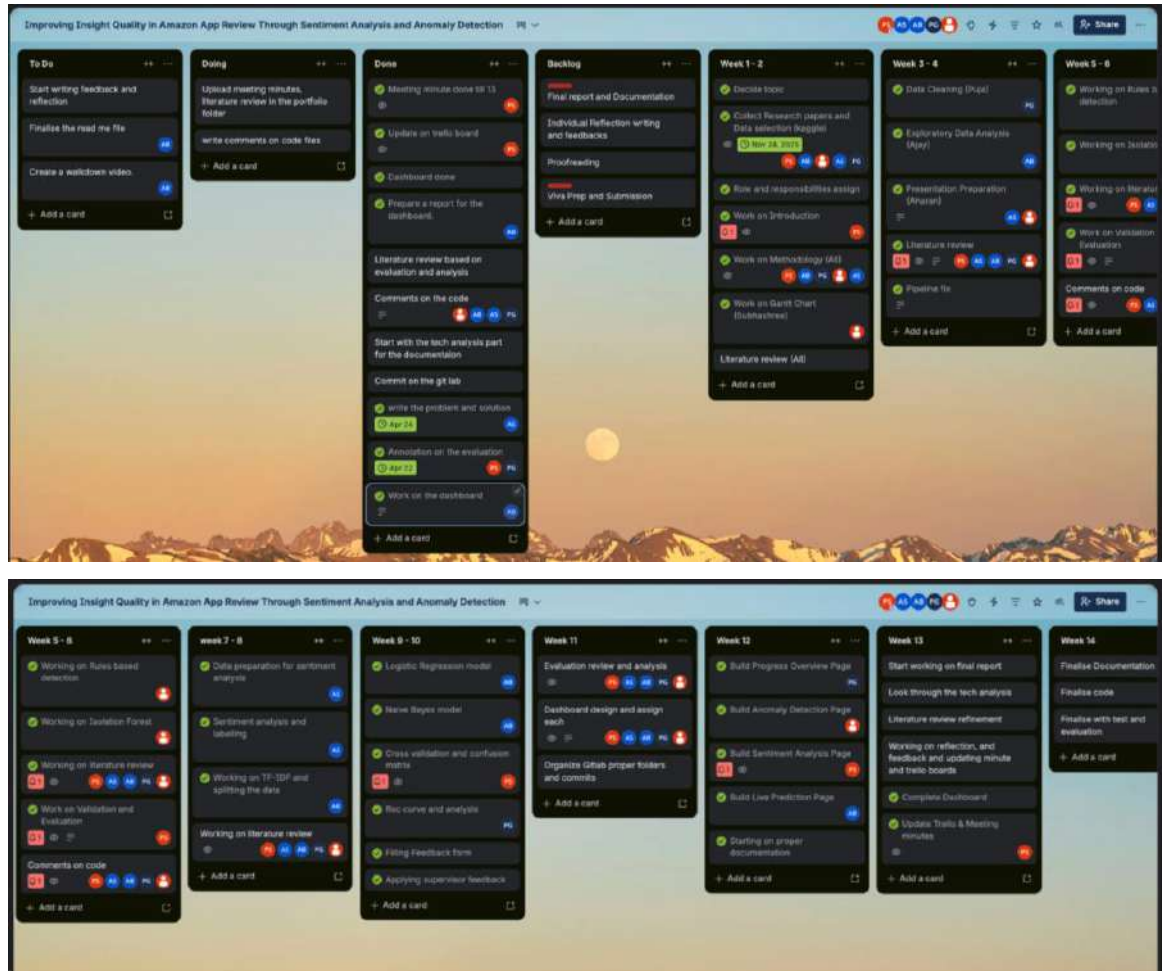
Liu, F.T., Ting, K.M. and Zhou, Z.-H. (2008) 'Isolation Forest', *Proceedings of the 2008 IEEE International Conference on Data Mining*, pp. 413–422.

Mukherjee, A., Venkataraman, V., Liu, B. and Glance, N. (2013) 'What Yelp fake review filter might be doing?', *Proceedings of the International AAAI Conference on Web and Social Media*, 7(1), pp. 409–418.

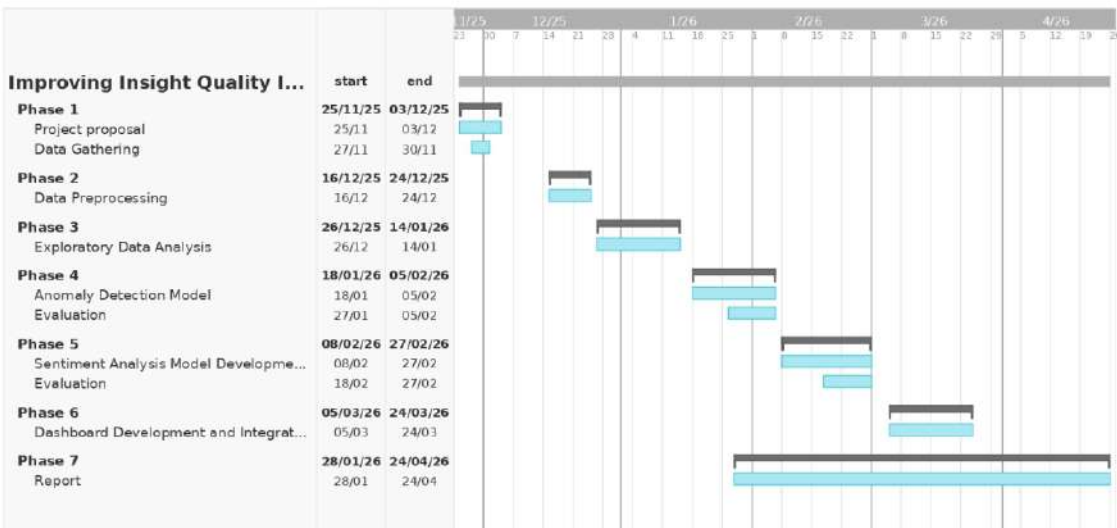
Pang, B., Lee, L. and Vaithyanathan, S. (2002) 'Thumbs up? Sentiment classification using machine learning techniques', *Proceedings of EMNLP 2002*, pp. 79–86.

13. Appendices:

A. Trello Screenshots



B. Gantt Chart



C. Meeting Minutes

IGP_sep25_Team11 / portfolio

To exit full screen, press and hold esc

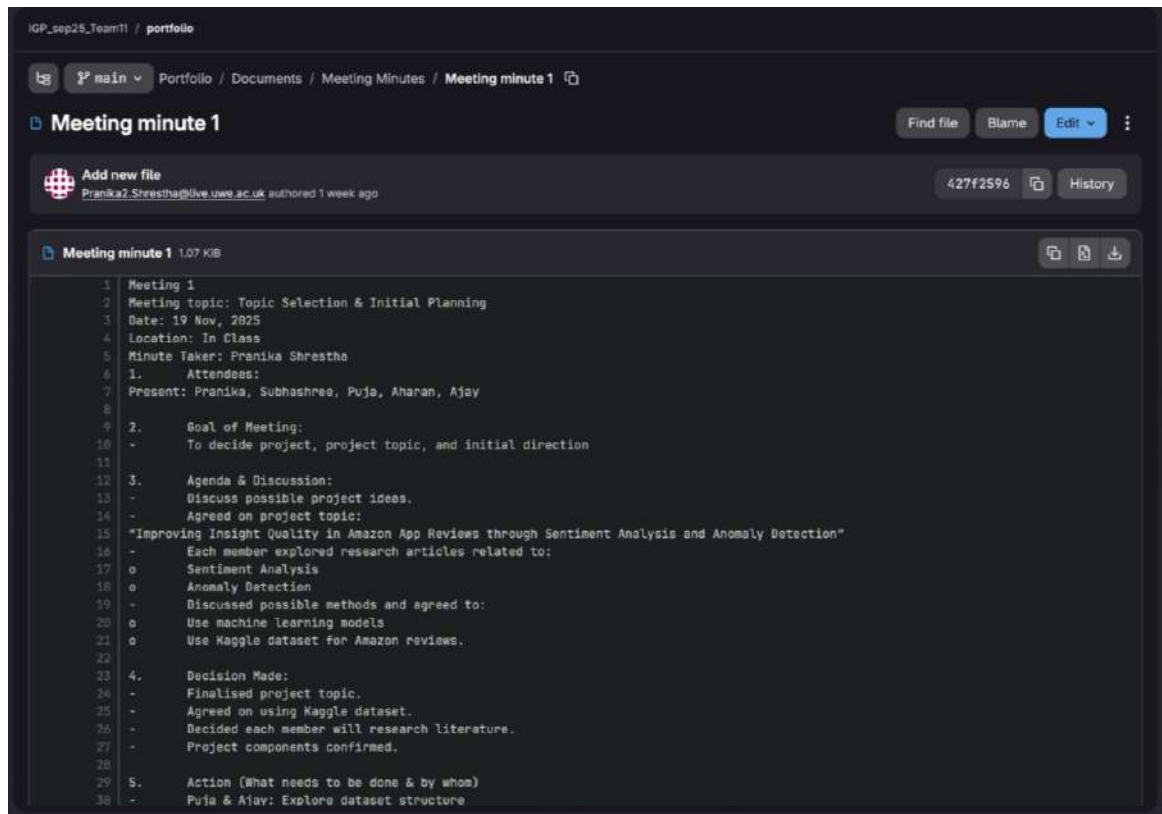
Edited Meeting minute 13

Pranika2.Shrestha@live.uwe.ac.uk authored 1 week ago

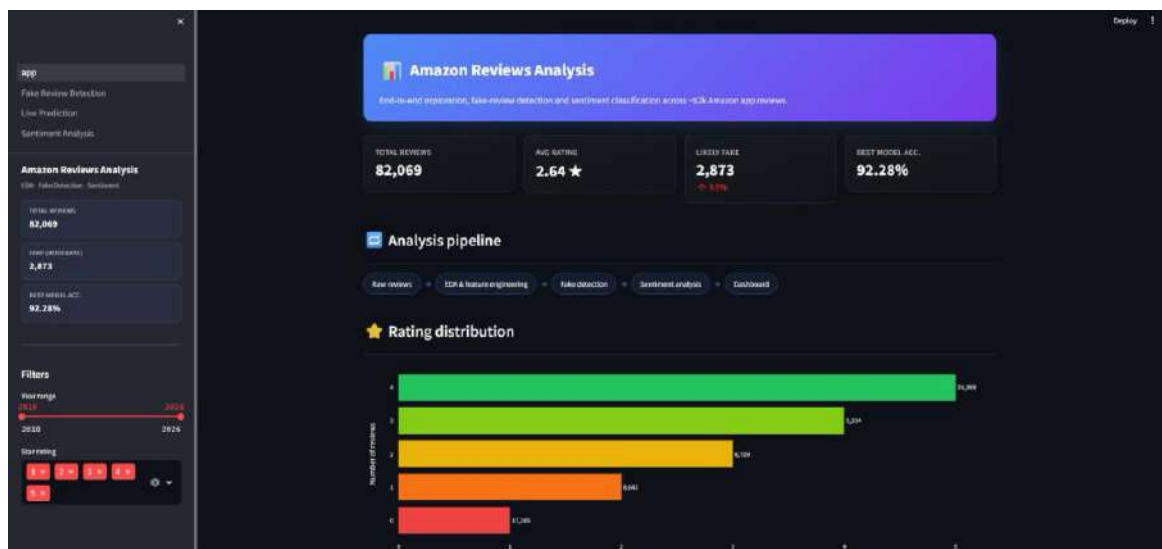
9470eb24

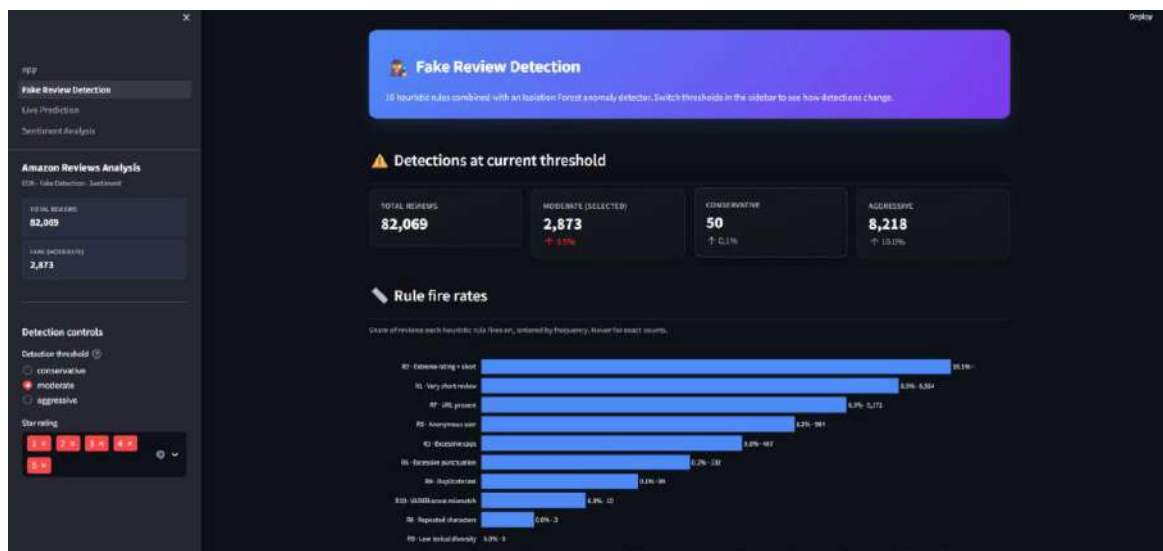
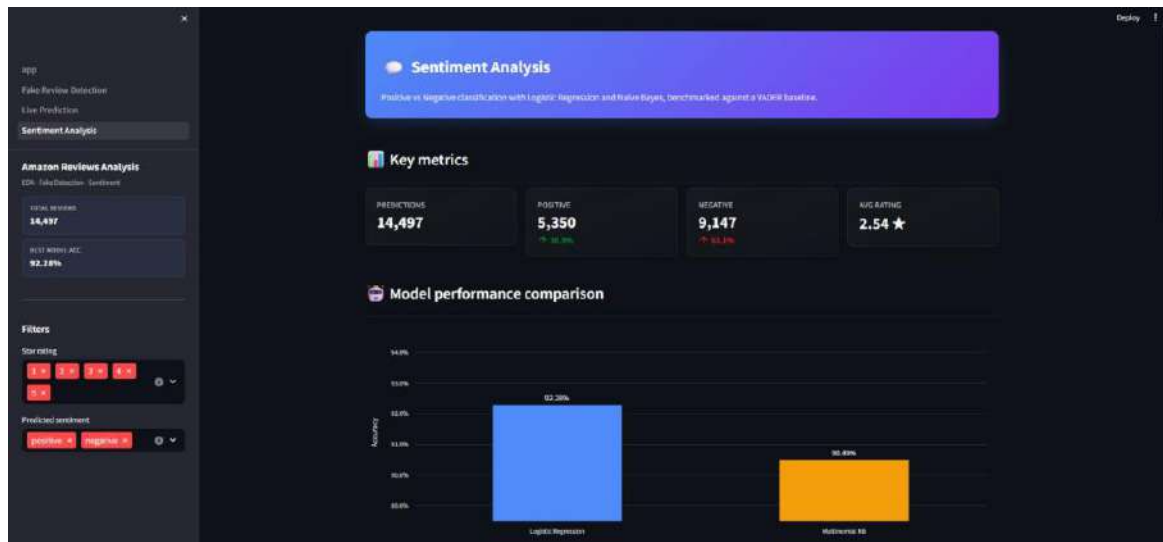
History

Name	Last commit	Last update
...		
.gitkeep	This includes all the meetings minute recorded since the beginning of this project.	1 week ago
Meeting minute 1	Add new file	1 week ago
Meeting minute 10	Adding meeting minute 10	1 week ago
Meeting minute 11	Adding meeting minute 11	1 week ago
Meeting minute 12	Adding meeting minute 12	1 week ago
Meeting minute 13	Edited Meeting minute 13	1 week ago
Meeting minute 2	meeting minute 2	1 week ago
Meeting minute 3	meeting minute 3	1 week ago
Meeting minute 4	meeting minute 4	1 week ago
Meeting minute 5	meeting minute 5	1 week ago
Meeting minute 6	meeting minute 6	1 week ago
Meeting minute 7	meeting minute 7	1 week ago
Meeting minute 8	meeting minute 8	1 week ago
Meeting minute 9	meeting minute 9	1 week ago

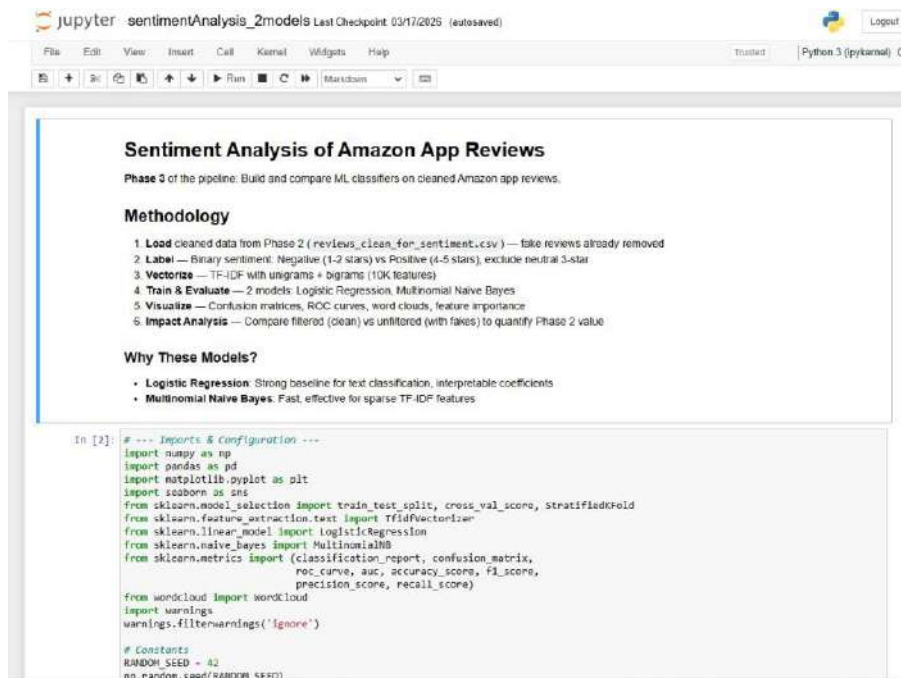


D. Dashboard Screenshots





E. Notebook Screenshots



Sentiment Analysis of Amazon App Reviews

Phase 3 of the pipeline: Build and compare ML classifiers on cleaned Amazon app reviews.

Methodology

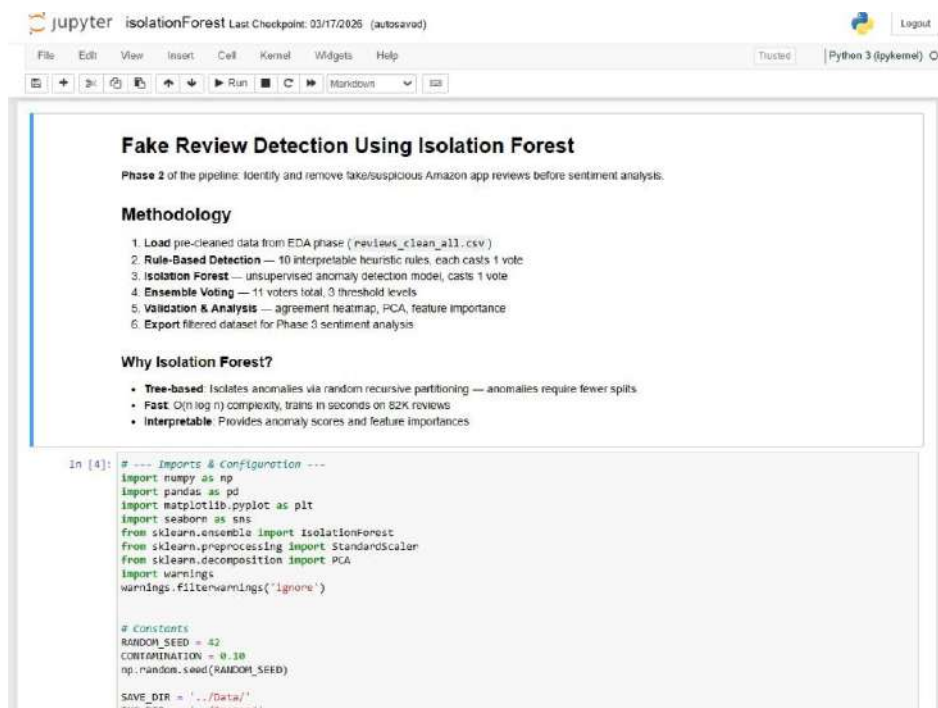
1. **Load** cleaned data from Phase 2 (reviews_clean_for_sentiment.csv) — fake reviews already removed
2. **Label** — Binary sentiment: Negative (1-2 stars) vs Positive (4-5 stars), exclude neutral 3-star
3. **Vectorize** — TF-IDF with unigrams + bigrams (10K features)
4. **Train & Evaluate** — 2 models: Logistic Regression, Multinomial Naive Bayes
5. **Visualize** — Confusion matrices, ROC curves, word clouds, feature importance
6. **Impact Analysis** — Compare filtered (clean) vs unfiltered (with fakes) to quantify Phase 2 value

Why These Models?

- **Logistic Regression**: Strong baseline for text classification, interpretable coefficients
- **Multinomial Naive Bayes**: Fast, effective for sparse TF-IDF features

```
In [2]: # --- Imports & Configuration ---
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, cross_val_score, StratifiedKFold
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import classification_report, confusion_matrix,
                                roc_curve, auc, accuracy_score, f1_score,
                                precision_score, recall_score
from wordcloud import WordCloud
import warnings
warnings.filterwarnings('ignore')

# Constants
RANDOM_SEED = 42
np.random.seed(RANDOM_SEED)
```



Fake Review Detection Using Isolation Forest

Phase 2 of the pipeline: Identify and remove fake/suspicious Amazon app reviews before sentiment analysis.

Methodology

1. Load pre-cleaned data from EDA phase (reviews_clean_all.csv)
2. **Rule-Based Detection** — 10 interpretable heuristic rules, each casts 1 vote
3. **Isolation Forest** — unsupervised anomaly detection model, casts 1 vote
4. **Ensemble Voting** — 11 voters total, 3 threshold levels
5. **Validation & Analysis** — agreement heatmap, PCA, feature importance
6. **Export** filtered dataset for Phase 3 sentiment analysis

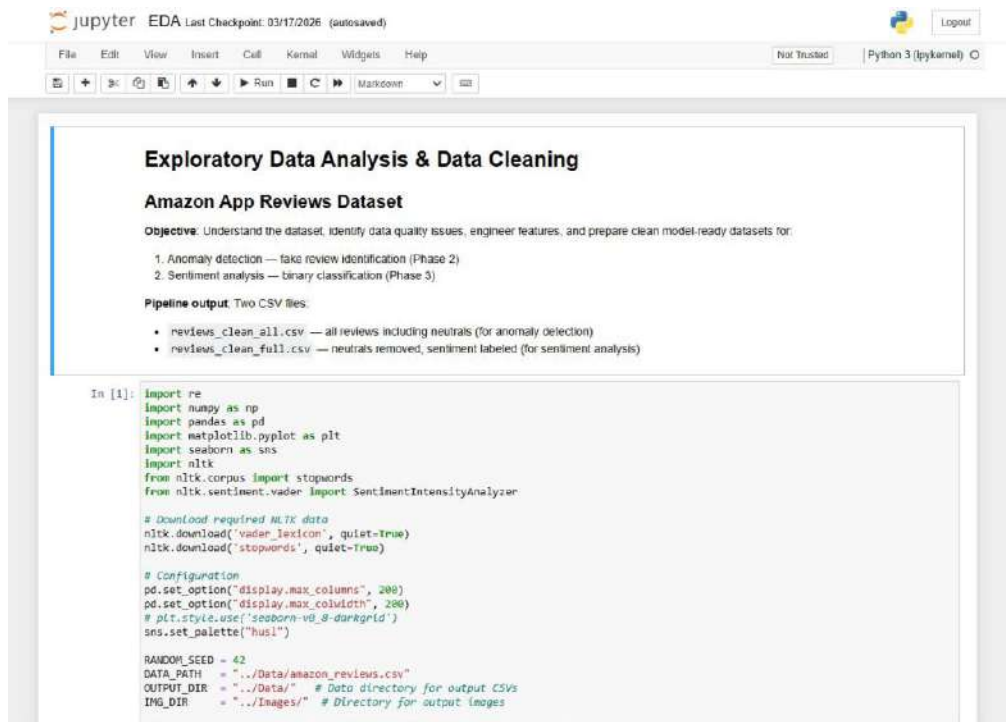
Why Isolation Forest?

- **Tree-based**: Isolates anomalies via random recursive partitioning — anomalies require fewer splits
- **Fast**: $O(n \log n)$ complexity, trains in seconds on 82K reviews
- **Interpretable**: Provides anomaly scores and feature importances

```
In [4]: # --- Imports & Configuration ---
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.ensemble import IsolationForest
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
import warnings
warnings.filterwarnings('ignore')

# Constants
RANDOM_SEED = 42
COMBINATION = 0.30
np.random.seed(RANDOM_SEED)

SAVE_DIR = '../Data/'
IMG_DIR = '../Images/'
```



jupyter EDA Last Checkpoint: 03/17/2026 (autosaved) Logout

File Edit View Insert Cell Kernel Widgets Help Not Trusted Python 3 (ipykernel)

Run Markdown

Exploratory Data Analysis & Data Cleaning

Amazon App Reviews Dataset

Objective: Understand the dataset, identify data quality issues, engineer features, and prepare clean model-ready datasets for:

1. Anomaly detection — fake review identification (Phase 2)
2. Sentiment analysis — binary classification (Phase 3)

Pipeline output: Two CSV files:

- `reviews_clean_all.csv` — all reviews including neutrals (for anomaly detection)
- `reviews_clean_full.csv` — neutrals removed, sentiment labeled (for sentiment analysis)

```
In [1]: import re
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import nltk
from nltk.corpus import stopwords
from nltk.sentiment.vader import SentimentIntensityAnalyzer

# Download required NLTK data
nltk.download('vader_lexicon', quiet=True)
nltk.download('stopwords', quiet=True)

# Configuration
pd.set_option("display.max_columns", 200)
pd.set_option("display.max_colwidth", 200)
# plt.style.use('seaborn-v0_8-darkgrid')
sns.set_palette("husl")

RANDOM_SEED = 42
DATA_PATH = "../Data/amazon_reviews.csv"
OUTPUT_DIR = "../Data/" # Data directory for output CSVs
IMG_DIR = "../Images/" # Directory for output images
```

Declaration of Writing Support: Sudicity was used only to receive formative feedback on selective section of portfolio, such as reflection and literature review structure, grammar, and clarity, the final content, interpretation, analysis, and working were reviewed and completed by the team.